

Business Requirements Document

Restaurant Operations Platform (working title)

Field	Value
Document	Business Requirements Document (BRD)
Version	1.0, baseline for development
Date	25 September 2026
Business Owner	Kunal & Kshitij
Intended reader	Senior Software Developer / Development Team Lead
Status	Draft for sign-off
Inputs	(1) "Restaurant POS platform: development blueprint" (requirements.docx); (2) Business Owner's product brief; (3) requirements workshop of 25 Sep 2026 (decisions recorded in Appendix A)

Revision history

Version	Date	Author	Change
1.0	25 Sep 2026	Business Owner (drafted with Claude)	First baseline after requirements workshop

How to read this document

- **Normative words.** *Shall*= mandatory. *Should*= strongly recommended; deviation must be justified. *May*= optional.
- **Priorities (MoSCoW).**
 - **M:** Must have for the first commercial release (v1.0).
 - **S:** Should have. In v1.0 unless the team raises a schedule risk, in which case it moves to the first follow-up release with the Business Owner's approval.

- **C:** Could have / future scope. Not built in v1.0, but the design **shall not preclude** it.
- **Requirement IDs** follow AREA-NNN (e.g., ORD-003). IDs are permanent: never renumber; mark withdrawn requirements as *Withdrawn*.
- ⚙️ **marks a configurable default**, not a hard-coded constant. The value shown is the factory default; the restaurant (or the vendor, where stated) can change it.
- **Technology.** Where this BRD names a technology, the development lead may substitute an equivalent if the requirement it serves is still met. Every substitution is recorded as an Architecture Decision Record (ADR) in the repository.
- **"Vendor"** means the Business Owner's company, which builds, sells and supports the platform. **"Restaurant"** means a customer of the vendor.

Table of contents

1. [Executive summary](#)
2. [Business objectives and success metrics](#)
3. [Scope](#)
4. [Stakeholders and user roles](#)
5. [Solution overview](#)
6. [Functional requirements](#)
7. [Non-functional requirements](#)
8. [Security requirements](#)
9. [Data requirements](#)
10. [Technical architecture and recommended stack](#)
11. [Hardware and site requirements](#)
12. [Compliance and regulatory requirements \(India\)](#)
13. [Assumptions](#)
14. [Constraints](#)
15. [Dependencies](#)
16. [Risks and mitigations](#)
17. [Release plan](#)
18. [Testing and acceptance](#)
19. [Business Owner responsibilities](#)
20. [Open items](#)
21. [Glossary](#)

- [Appendix A: Decision log](#)
 - [Appendix B: State machines](#)
 - [Appendix C: Default notification matrix](#)
 - [Appendix D: Changes from the original blueprint](#)
-

1. Executive summary

1.1 The problem

Many dine-in restaurants still run service on paper order slips (KOTs), verbal hand-offs and waiters walking back and forth between the floor, the kitchen and the cash counter. This causes:

- slow and error-prone order transfer to the kitchen;
- food waiting at the pass because the right waiter doesn't know it is ready;
- missed or slow responses to diners who want water, a waiter or the bill;
- little systematic upselling;
- weak visibility for owners into who did what, including revenue leakage through deleted or edited bills and items sent to the kitchen but never billed.

1.2 The solution

A **locally hosted, integrated restaurant operations platform** that digitises the full order cycle, from the diner's table to the kitchen and back to the bill. It has these components:

#	Component	What it does
1	Main POS and Manager Console (Windows desktop application + local server + local database)	Billing, table management, manager dashboard, reports, configuration. The heart of the system; installed from a single executable.
2	Kitchen Display System (KDS)	Shows orders to each kitchen station; marks items preparing/ready; prints kitchen tickets where configured.
3	Waiter (Captain) App (Android APK)	Waiters take orders, approve customer orders, receive alerts, manage their tables.

4	Customer Table Tablet (Android, kiosk mode)	Diners browse the menu, get recommendations, place orders (approved by the waiter), call for water/waiter/bill, track order status.
5	QR Online Menu (public website)	Per-table QR code opens the live menu on the diner's phone; dine-in ordering (approved by the waiter). Always in sync with the restaurant's menu.
6	Wrist Pager (ESP32 wearable)	Vibrates and displays short messages ("T7 READY", "T4 WATER") to the right waiter, with an acknowledge button.
7	Recommendation Engine	Suggests what to order next: restaurant-defined rules, then patterns learned from the restaurant's own orders (machine learning), then best sellers.
8	Vendor Control Plane (vendor's cloud)	Restaurant onboarding, licenses, subscription enforcement, remote updates (per restaurant if needed), fleet health monitoring, backups.

Core design principle: local-first. Everything that happens inside the restaurant (ordering, kitchen, pagers, billing, reports) works **without internet**. The cloud is used only for what must cross the internet: QR orders from diners' phones, licensing, updates, off-site backups and vendor monitoring.

1.3 Business value

- **Faster service.** Orders reach the correct kitchen station instantly; the right waiter is paged the moment food is ready or a table needs attention.
- **Higher average bill.** Recommendations on the tablet, QR menu and waiter app.
- **Control and trust.** Every action is attributed to a person and device. Bills can't be deleted, voids and discounts need approval, and a daily suspicious-activity report goes to the owner.
- **Low running cost for the restaurant.** Data stays on its own PC; the kitchen display runs on inexpensive Android hardware.
- **Recurring revenue for the vendor.** Subscription model with remote license control and remote updates.

1.4 Key decisions (full log in Appendix A)

- One outlet per owner. The owner accesses reports on the restaurant's system, or staff export and send them. No cloud owner dashboard in v1.
- The platform is sold **as a complete ecosystem only** in v1. The code is nevertheless built as isolated modules with documented interfaces, so components (tablet, pager, QR menu) can later be integrated with third-party POS systems and sold separately.
- **Every customer-originated order (tablet or QR) requires waiter approval** before it reaches the kitchen; the waiter is notified.
- **Unacknowledged alerts escalate directly to the manager** after a configurable time.
- **No payment integration.** Staff record the payment mode (cash, card, UPI, other) manually.
- English only. India only. Dine-in and takeaway.
- The license "kill switch" is **staged** (reminders → grace period → restricted mode). It never interrupts a service in progress, and never deletes or withholds the restaurant's data.

1.5 Scope at a glance

In v1: all eight components above, GST-compliant billing, anti-fraud audit trail, backup/archive, onboarding wizard with menu import, remote updates and licensing.
Out of v1: payments integration, aggregator (Zomato/Swiggy) integration, inventory/recipes, multi-outlet, loyalty, non-English UI, third-party POS integration (architecture-ready only).
Details in [section 3](#).

2. Business objectives and success metrics

ID	Business objective
BO-1	Reduce the time between a diner deciding what to order and the kitchen starting to prepare it.
BO-2	Reduce the time food waits at the pass and the time diners wait for service requests.
BO-3	Increase average order value through recommendations.
BO-4	Give owners full accountability for every order, discount, cancellation and bill.

BO-5	Make onboarding a new restaurant fast and repeatable, with its own custom menu.
BO-6	Build a recurring-revenue business the vendor can operate, update and enforce remotely at scale.

Success metrics. Targets are proposals to be validated during the pilot. Pre-pilot baselines are measured by the Business Owner (see § 19).

KPI	Target	Objective	How measured
Approved order → visible on KDS	≤ 1 s (95th percentile)	BO-1	System timestamps
Kitchen "Ready" → pager vibrates	≤ 2 s (95th percentile)	BO-2	System timestamps
Service request → waiter acknowledgement	median ≤ 60 s	BO-2	Notification ack times (RPT-003)
Ready → picked up	≥ 25 % reduction vs. pilot baseline	BO-2	RPT-010
Customer order waiting for waiter approval	median ≤ 90 s	BO-1	Approval timestamps
Recommendation attach rate	≥ 8 % of tablet/QR orders contain a recommended item	BO-3	RPT-012
Voids, discounts, reprints attributed to a person	100 %	BO-4	Audit log
New restaurant live (≤ 150 items, ≤ 40 tables)	≤ 1 business day	BO-5	Onboarding records
Local system availability during trading hours	≥ 99.9 % (excluding site power/hardware failure)	BO-6	Heartbeats
Open Critical/High security findings at go-live	0	BO-6	Penetration test report

New waiter proficient	≤ 30 min training	BO-5	Pilot usability test
-----------------------	-------------------	------	----------------------

3. Scope

3.1 In scope for v1.0

1. **Local server and database** on the restaurant's Windows PC, installed by a single signed installer; data on a selectable drive (default: non-system drive such as D:).
2. **POS and Manager Console:** table management, order entry, billing (GST), manual payment recording, shift/day-end, manager dashboard, reports and exports, configuration, audit viewer.
3. **Kitchen Display System** (browser-based) with station routing and thermal kitchen-ticket printing.
4. **Waiter App** (Android APK).
5. **Customer Table Tablet App** (Android, kiosk mode): menu, search (text; voice as S), recommendations, ordering with waiter approval, service buttons (Water, Waiter, Bill, Cancel), live order status, feedback.
6. **QR Online Menu** with dine-in ordering through a cloud relay, waiter approval and customer-visible status.
7. **Wrist Pager:** hardware specification, firmware, pairing, alerts, acknowledgement, OTA updates.
8. **Notification and escalation engine** (configurable rules).
9. **Recommendation engine** (rules + learned + best sellers).
10. **Menu management:** categories, items, photos, variants, modifiers, combos, availability and stock counts, tags, synonyms.
11. **Audit trail and anti-fraud reporting.**
12. **Data lifecycle:** automated local and encrypted cloud backups, restore, archive-then-purge, storage monitoring.
13. **Licensing and subscription enforcement** (staged).
14. **Remote updates** for all components with channels, per-restaurant pinning, staged rollout and rollback.
15. **Vendor Control Plane** (cloud): tenants, licenses, subscriptions (manual payment recording), fleet monitoring, release management, support diagnostics, onboarding services.

16. **Onboarding:** setup wizard, Excel menu import, AI-assisted menu import (S), QR table-tent generation, device pairing.
17. **Takeaway orders** from the POS (and waiter app, S).
18. **Modular architecture** with documented contracts (integration-ready).

3.2 Out of scope for v1.0 (explicit)

Item	Note
Payment gateway, card terminal or UPI integration	Payment mode recorded manually. UPI QR with amount on the bill is future scope.
Online payment on QR orders	No prepayment; bill settled at the table/counter.
Multi-outlet / chain management, central menu, cloud owner dashboard	One outlet per owner.
Aggregator integration (Zomato, Swiggy, etc.)	Data model keeps <code>order source</code> and external IDs so it can be added.
Raw-material inventory, recipes, purchase management	Only item-level availability and simple stock counts are in scope.
Loyalty programmes, customer accounts, personalised recommendations	Recommendations are table-level only.
Delivery orders; takeaway ordering via QR	QR is dine-in only.
Languages other than English	UI strings externalised so languages can be added later.
Selling modules separately; integration with third-party POS	Architecture-ready (see INT requirements); not delivered.
iOS apps; macOS/Linux desktop	Android and Windows only.
Table reservations, staff attendance/payroll, accounting-software export (e.g., Tally)	Future.

GST e-invoicing (IRN) for B2B invoices	Customer GSTIN can be captured; e-invoicing applicability to be confirmed with the restaurant's CA.
Bar/liquor billing (VAT)	Excluded unless the pilot restaurant needs it (open item OI-03).
Bring-your-own-device for staff	Staff devices are restaurant-owned and managed (MDM).

3.3 Future scope (design must not preclude)

Per-module licensing and "module mode" installation; adapters for third-party POS systems; public integration API and webhooks; aggregator integration; payment integration (dynamic UPI QR, card terminals, payment gateway on QR); multi-outlet with cloud owner dashboard; loyalty and personalised recommendations; additional UI languages; inventory and recipe management; reservations; accounting export; hot-standby server PC; custom pager hardware.

4. Stakeholders and user roles

4.1 Stakeholders

Stakeholder	Interest / responsibility	Involvement
Business Owner (Kshitij)	Product vision, funding, commercial model, final sign-off on scope and releases	Approves BRD, decisions, releases; owns vendor accounts, contracts and compliance (§ 19)
Development Team (lead + backend, web, mobile, firmware, QA, DevOps)	Design, build, test, release, maintain	Owns architecture, ADRs, estimates, quality
Restaurant Owner (customer)	Buys the platform; wants control, insights, low cost, reliability	Data owner; approves sensitive actions (purge, restore); pilot feedback

Restaurant Manager	Runs daily operations	Primary admin user: staff, menu, devices, configuration, escalations, reports
Cashier	Billing and payments	Uses POS
Captain / Waiter	Takes and approves orders, serves, responds to requests	Uses waiter app and wrist pager
Kitchen staff / Chef	Prepares food	Uses KDS and kitchen printers
Diner (restaurant customer)	Wants fast, easy ordering and service	Uses table tablet or QR menu; no login
Vendor Onboarding and Support team	Installs, onboards, supports restaurants	Uses Vendor Control Plane; runs onboarding
Vendor Finance / Super Admin	Subscriptions, licenses, releases	Uses Vendor Control Plane
Hardware suppliers	Tablets, pagers, printers, routers	Supply per supported-hardware list
External advisers	Restaurant's CA (tax configuration), vendor's legal counsel, compliance consultant (hardware certification), penetration-testing firm	Advice and certification
Cloud and service providers	Supabase, hosting, Expo, MDM, messaging, monitoring	Contracted by the Business Owner

4.2 User roles and permissions

Human roles inside a restaurant: **Owner, Manager, Cashier, Waiter (Captain), Kitchen.**

Non-human actors: **Table tablet** (device bound to one table), **QR guest** (anonymous diner's browser), **Pager** (device bound to one waiter).

Legend: ✓ allowed • **PIN** allowed only with a manager's PIN override (both people recorded) • own = own shift/own tables only • - not allowed.

Capability	Owner	Manager	Cashier	Waiter	Kitchen
Open table, take order, send KOT	✓	✓	✓	✓	-
Approve / reject customer (tablet, QR) orders	✓	✓	✓	✓ (own)	-
Cancel item before preparation starts	✓	✓	✓	✓ (own)	-
Void item after preparation started	✓	✓	PIN	PIN	-
Mark item preparing / ready	✓	✓	-	-	✓
Mark item picked up / served	✓	✓	✓	✓	✓ (picked up)
Toggle item out-of-stock / set stock count ⚙️ (OI-11)	✓	✓	✓	-	✓
Move / merge tables	✓	✓	✓	✓ (own)	-
Print bill, record payment	✓	✓	✓	- (can request bill)	-
Discount within role limit ⚙️	✓	✓	✓	-	-
Discount above limit, complimentary item	✓	✓	PIN	-	-
Remove service charge (if enabled)	✓	✓	✓	-	-
Reprint bill (marked DUPLICATE)	✓	✓	✓	-	-
Edit bill after printing	✓	✓	PIN	-	-
Cancel (void) an invoice	✓	✓	PIN	-	-
Cash in/out, close shift	✓	✓	✓ (own)	-	-

Day-end close	✓	✓	-	-	-
Manage menu, prices, combos, photos	✓	✓	-	-	-
Manage staff, PINs, sections, pager assignment	✓	✓	-	-	-
Pair / unpair devices	✓	✓	-	-	-
Configure notifications, recommendations, printers, stations	✓	✓	-	-	-
Tax, invoice series, restaurant legal details	✓	-	-	-	-
View reports / export	✓	✓	own shift	-	-
View audit log	✓	✓	-	-	-
Backup restore, archive and purge, full data export	✓ (password + 2FA)	-	-	-	-
License and subscription screens	✓	view	-	-	-
Approve "install update now"	✓	✓	-	-	-

- **Table tablet:** view menu; submit orders *for approval* for its own table only; raise/cancel service requests for its own table; submit feedback.
- **QR guest:** view menu; submit orders *for approval* for the scanned table only; (S) raise service requests.
- **Pager:** receive its own alerts; acknowledge.

Permissions are enforced on the server for every request (AUTH-010). The Owner may create custom roles (AUTH-012, S).

5. Solution overview

5.1 Context diagram

```
VENDOR CLOUD (India region)
├ Vendor Control Plane: tenants, licenses, subscriptions, fleet
health,
|   release management, support diagnostics, AI menu import
├ Update distribution: Windows installer/updates, Android OTA + APKs,
|   pager firmware
├ QR relay: Edge functions + Postgres (RLS) + Realtime + Storage
|   (menu snapshot, photos, QR orders, restaurant presence)
└ Encrypted backup storage
    ▲
    |   outbound HTTPS/WSS only; nothing connects into the
restaurant
    |
RESTAURANT: staff network (separate SSID/VLAN from guest Wi-Fi)
├ Restaurant PC (Windows 11)
|   └ Local Server (Windows service): REST API, Socket.io, MQTT
broker,
|       notification engine, recommendation engine, sync agent,
|       license agent, backup agent, web console hosting
|       └ PostgreSQL (Windows service, data on D:\ or chosen drive)
|       └ Watchdog (restarts services)
|       └ POS and Manager Console (Electron shell around the web console)
├ KDS screens (browser on Android tablet/TV) + thermal printers
├ Waiter App (Android phones, MDM-managed)
├ Table Tablets (Android, kiosk mode, MDM-managed)
├ Manager's phone/laptop browser (paired) for the dashboard
└ Wrist Pagers (ESP32, MQTT over TLS)

DINER'S PHONE → QR Online Menu (public site) → QR relay → Local Server
```

5.2 Components

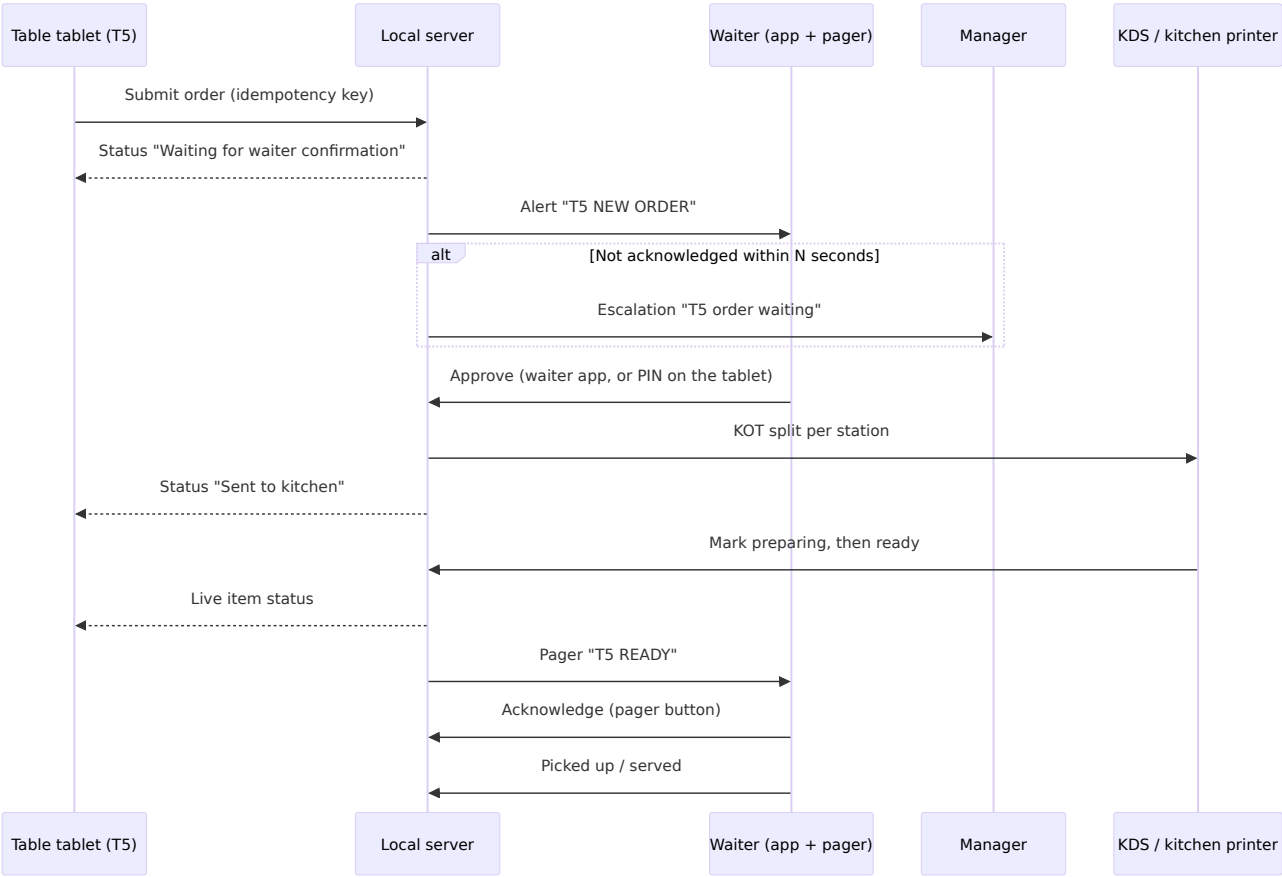
ID	Component	Runs on	Primary users	Recommended technology (see § 10)

C1	Local Server	Restaurant PC, as Windows service	All apps	NestJS (Node.js LTS), Prisma, Socket.io, Aedes MQTT
C2	Local Database	Restaurant PC, as Windows service	C1 only	PostgreSQL
C3	Web Console (POS, Manager Dashboard, KDS modes)	Served by C1; opened in Electron on the PC or in a browser on paired devices	Owner, Manager, Cashier, Kitchen	React + TypeScript
C4	Desktop Shell	Restaurant PC	Cashier, Manager	Electron (electron-builder, electron-updater)
C5	Waiter App	Android phones	Waiters	React Native + Expo
C6	Table Tablet App	Android tablets (kiosk)	Diners, waiters (approval at table)	React Native + Expo
C7	QR Online Menu	Public web hosting	Diners	Next.js
C8	QR Relay	Vendor cloud	C7, C1	Supabase (Postgres + RLS, Edge Functions, Realtime, Storage)
C9	Wrist Pager Firmware	ESP32-S3 wearable	Waiters, managers	C/C++ (ESP-IDF), esp-mqtt
C10	Recommendation Engine	Module inside C1; snapshot published to C8	C5, C6, C7	TypeScript (in-process), nightly job
C11	Vendor Control Plane	Vendor cloud	Vendor staff	NestJS + React, managed Postgres (India region)

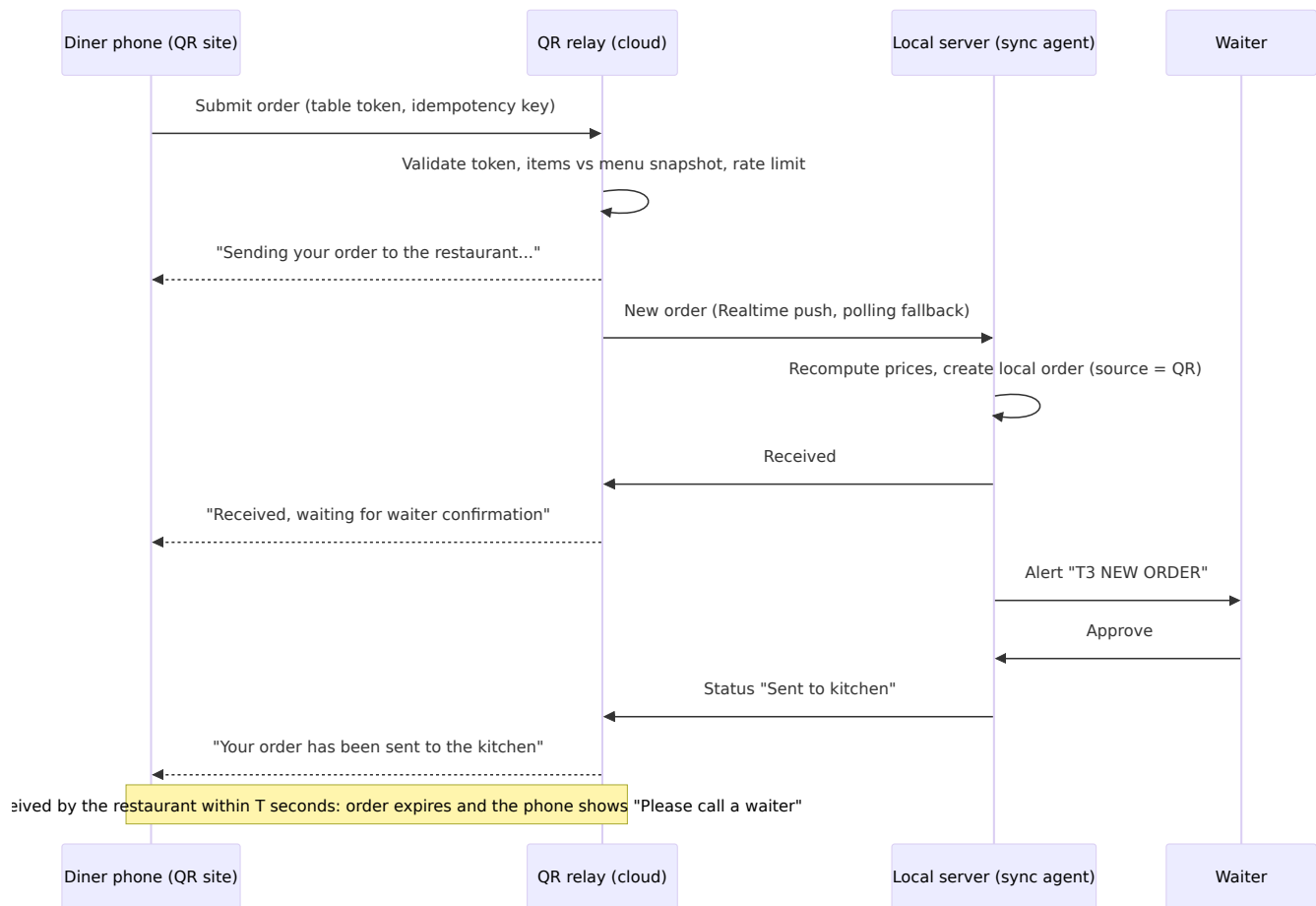
C12	Installer, Updater, Watchdog	Restaurant PC	Vendor/IT	electron-builder / NSIS, service wrapper
-----	------------------------------	---------------	-----------	--

5.3 Key end-to-end flows

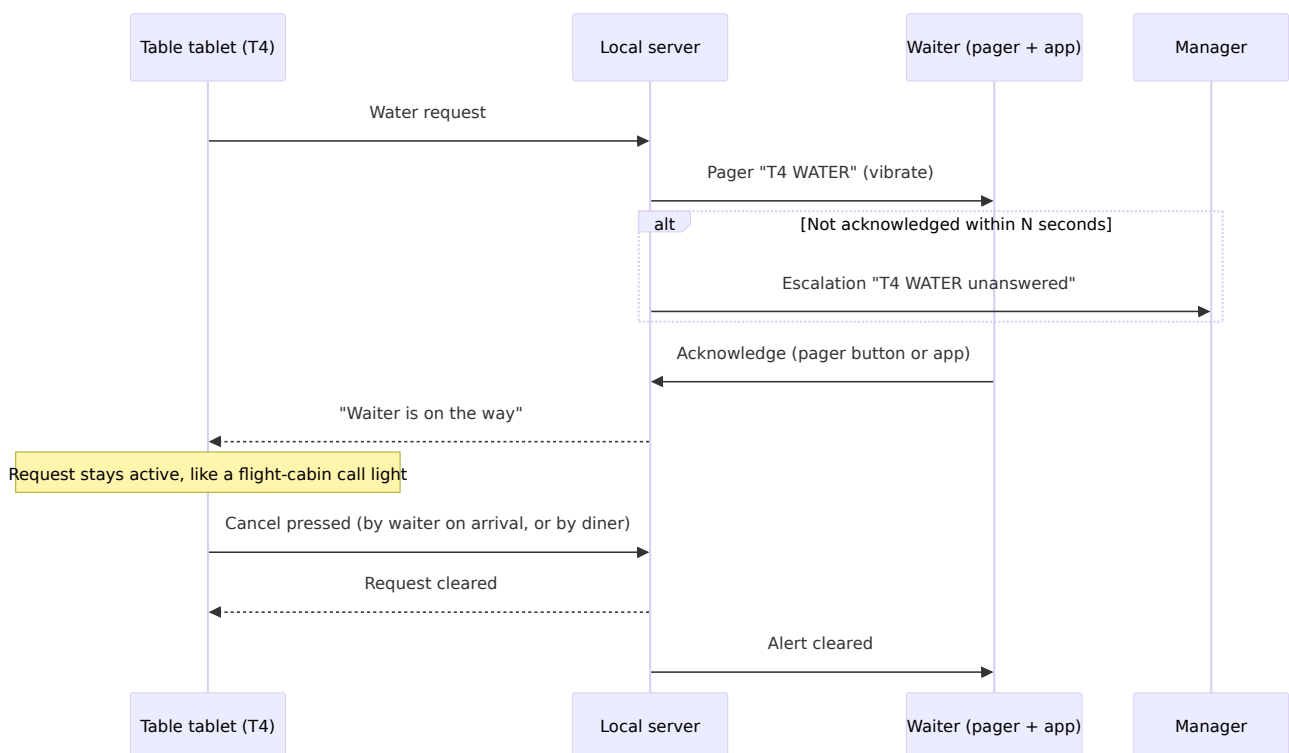
F1: Table tablet order with waiter approval, kitchen, pager.



F2: QR order through the cloud relay.



F3: Service request (Water / Waiter / Bill) with acknowledgement, escalation and Cancel.



F4: Billing. Diner presses *Bill* (or waiter requests) → cashier and waiter notified → cashier reviews the running order, applies permitted discounts, prints the GST invoice → payment mode(s) recorded → table closed → tablet wiped and reset → table shows Free.

6. Functional requirements

Each requirement is written as **ID (priority)**, followed by the requirement. Items marked "(blueprint)" come from the original development blueprint; everything else was added during the requirements workshop.

6.1 Authentication, sessions and device pairing (AUTH)

- **AUTH-001 (M)** Staff log in on shared devices by **selecting their name/photo tile, then entering a numeric PIN** (4 digits by default; the owner may require 6 ✨). Choosing a name first means PINs need not be unique, and setting a PIN can never reveal someone else's.
- **AUTH-002 (M)** PINs shall never be stored, transmitted in logs or displayed in plain text. They shall be hashed with bcrypt (cost ≥ 12) or Argon2id (blueprint). Before hashing, they are combined with a server-side secret ("pepper") kept outside the database and protected by Windows DPAPI. A 4-digit PIN has only 10,000 combinations, so without the pepper a stolen database could be brute-forced in seconds.
- **AUTH-003 (M)** After **5 failed attempts within 10 minutes** (blueprint) a staff login is locked for 15 minutes ✨ or until a manager unlocks it. Attempts are also rate-limited per device.
- **AUTH-004 (M)** One reusable PIN-pad component is used for login and manager overrides across the POS, KDS, waiter app and table tablet (blueprint).
- **AUTH-005 (M)** On successful login the server issues a short-lived signed access token (≤ 15 min ✨) bound to staff member, role, restaurant and device, plus a refresh mechanism (blueprint: JWT).
 - Sessions expire after inactivity (POS and waiter devices: 10 min ✨) and are renewed automatically while the person is actively using the app (blueprint).
 - KDS screens run in device-authenticated *station mode*: actions are attributed to the station. Individual kitchen logins can be switched on ✨.

- **AUTH-006 (M)** The **Owner** account uses a strong password (≥ 12 characters) plus a TOTP second factor, which works offline. Both are required for: archive and purge, backup restore, full data export, license/subscription screens, tax and invoice settings, and creating or removing managers.
- **AUTH-007 (M) Device pairing.** Every device (KDS screen, waiter phone, table tablet, pager, manager's browser) must be paired by a manager before it can talk to the server.
 1. The manager generates a one-time pairing QR code (valid 10 min ☼).
 2. The device presents it.
 3. The server issues a per-device credential (key pair or client certificate), stored in the device's secure storage (Android Keystore, ESP32 encrypted storage, browser non-exportable key).

Requests from unpaired devices are rejected even when a valid PIN is entered.

- **AUTH-008 (M)** Unpairing a device, or deactivating a staff member, revokes their tokens and closes their live connections within 5 seconds.
- **AUTH-009 (M)** Each table tablet is paired to one table; re-assigning it requires a manager PIN. A tablet can only read or act on its own table.
- **AUTH-010 (M)** Role-based access control per § 4.2 is enforced **on the server** for every REST call, socket event and MQTT topic. Hiding buttons in the UI is not a security control.
- **AUTH-011 (M)** Manager override: actions marked **PIN** in § 4.2 prompt for a manager's PIN on the current device. The audit log records both the requester and the approver.
- **AUTH-012 (S)** The Owner can create custom roles by combining permissions.
- **AUTH-013 (M)** Logins, logouts, failed attempts, lockouts and overrides are written to the audit log (AUD).
- **AUTH-014 (M)** Vendor staff have **no standing access** to a restaurant's local system (see UPD-008, UPD-009).

6.2 Installation and onboarding (ONB)

- **ONB-001 (M)** A single signed Windows installer sets up:
 - the Local Server and PostgreSQL as auto-starting Windows services;
 - the watchdog;
 - the POS/Manager desktop app.

No manual database setup is needed (blueprint).

- **ONB-002 (M) Data drive selection.** The installer lists local fixed drives with free space and defaults to the first non-system drive (e.g., D:) if one exists. Otherwise it uses C: and shows a warning. It refuses network and removable drives and requires ≥ 20 GB  free space.
- **ONB-003 (M)** Activation requires a one-time activation code issued from the Vendor Control Plane. It binds the license to the restaurant and the PC (LIC-004).
- **ONB-004 (M)** The **first-run setup wizard** is usable by a non-technical manager, saves progress and can be resumed. Steps:
 1. **Restaurant profile:** legal name, display name, address, state, GSTIN, FSSAI license number, logo, contact, business hours, business-day cut-off time (default 04:00 .
 2. **Tax setup:** tax groups (e.g., "GST 5 %" = CGST 2.5 % + SGST 2.5 %; rates entered by the restaurant, never hard-coded), SAC code, prices tax-inclusive or tax-exclusive , rounding rule.
 3. **Invoice settings:** series prefix, financial-year reset, header/footer, service charge (off by default).
 4. **Accounts:** Owner (password + 2FA), managers.
 5. **Floor:** sections and tables (number/name, capacity). Visual drag-and-drop layout is S.
 6. **Stations and printers:** kitchen stations; network printer auto-discovery (S) or manual IP (blueprint); test print; station $\leftrightarrow$ printer mapping; per-station mode screen / print / both (blueprint).
 7. **Menu import** (ONB-005, ONB-006).
 8. **Staff and PINs**, section assignment.
 9. **Device pairing:** tablets, waiter phones, KDS screens, pagers.
 10. **QR codes:** generate and print table tents (PDF, A6/A5).
 11. **Go-live check:** a guided test order through every path (waiter, tablet, QR, KDS, pager, bill) before going live.
- **ONB-005 (M) Excel/CSV menu import** using a vendor-provided template: categories, items, prices, tax group, veg/non-veg/egg, tags, station, description, synonyms. Variants, modifiers and combos go in companion sheets. A validation report lists every error before anything is committed.
- **ONB-006 (S) AI-assisted menu import.** Upload photos or a PDF of the printed menu. A Vendor Control Plane service extracts it into the template structure. A **mandatory human**

review screen shows every item before commit. Requires internet; the AI provider's API key is held only in the vendor cloud.

- **ONB-007 (S)** Bulk photo upload, matched automatically to items by file name or item code, with manual correction.
- **ONB-008 (M)** Onboarding can be done on-site or remotely. Remote onboarding uses a screen-sharing session **started by the restaurant** (UPD-009).
- **ONB-009 (M)** Target: a restaurant with ≤ 150 items and ≤ 40 tables is fully operational within one business day, with the menu imported in ≤ 1 hour.
- **ONB-010 (S)** Export and import of settings, for moving to a new PC or reinstalling.
- **ONB-011 (M)** A printable **site-readiness checklist** covering PC spec, UPS, staff Wi-Fi separated from guest Wi-Fi, 2.4 GHz coverage in kitchen and floor, printer network, tablet offline speech pack (TAB-007) and BitLocker (SEC-007).

6.3 Menu management (MENU)

- **MENU-001 (M)** Structure: Menu $\rightarrow$ Categories (ordered, one level of sub-categories) $\rightarrow$ Items.
- **MENU-002 (M)** Item attributes:
 - name, short code, description, photo, base price, tax group;
 - food type (veg / non-veg / egg), spice level (0-3), tags (Jain, bestseller, chef's special, allergens, custom);
 - kitchen station, estimated prep time;
 - availability, optional stock count, display order;
 - visibility per channel (POS, waiter app, tablet, QR);
 - external ID (INT-005).
- **MENU-003 (M) Variants** (e.g., Half/Full, Regular/Large), each with its own price.
- **MENU-004 (M) Modifier groups** (e.g., "Roti type", "Spice level", "Add-ons") with required/optional, minimum/maximum selections and price deltas (+/-). A group can be reused across many items (blueprint).
- **MENU-005 (M) Combos**: a bundle of existing items and/or choice slots (e.g., "any 1 beverage") at a fixed price, with an optional active date range and time window for festivals or seasonal promotions (blueprint). Kitchen tickets always show each component item separately (blueprint).
- **MENU-006 (M) Availability and stock** (blueprint):

- Authorised roles (§ 4.2) can mark an item out of stock, or set a remaining quantity. The quantity decreases with each approved order and the item switches to out of stock at 0.
- Changes reach every in-restaurant device within 2 s, and the QR menu within 30 s when online. Items become non-orderable immediately.
- **MENU-007 (S)** Time-based availability (e.g., breakfast items 07:00-11:00 ⚙).
- **MENU-008 (M)** Photos are stored locally, auto-resized into standard renditions (WebP) and served to in-restaurant devices. They are mirrored to cloud storage for the QR menu (blueprint). Uploads up to 5 MB, compressed automatically; metadata (EXIF) stripped.
- **MENU-009 (M)** Price changes apply only to new order lines; open orders keep the price at the time of ordering. Every price change is audited.
- **MENU-010 (M)** Items that have ever been ordered cannot be deleted, only archived (hidden), to preserve reports.
- **MENU-011 (M)** Each item can have **search synonyms** (e.g., "panir", "cottage cheese"), used by text and voice search.
- **MENU-012 (M)** Every ordering surface (POS, waiter app, tablet, QR menu) uses the **same menu data and the same variant/modifier/combo selection logic and UI behaviour**, built from shared components (blueprint).
- **MENU-013 (M)** Menus are versioned. Tablets and waiter apps cache the current menu locally and refresh when a change event arrives, or on reconnection.

6.4 Tables, sections and sessions (TBL)

- **TBL-001 (M)** Sections (e.g., Ground floor, Terrace) contain tables. A table has a number/name, capacity, section, QR code and, optionally, a paired tablet.
- **TBL-002 (M) Waiter assignment:** at shift start the manager assigns waiters to sections, and optionally to individual tables. The **responsible waiter** for a table is its assigned waiter; this can be overridden per table session.
- **TBL-003 (M) Table session:** a waiter (app) or cashier/manager (POS) opens a table, which records seated-at time, number of guests (covers) and responsible waiter, and unlocks the table tablet. The session ends when the bill is settled and the table closed; the tablet then resets (TAB-003).
- **TBL-004 (M)** Table states: Free → Occupied → Bill requested → Bill printed → Settled/Free. "Needs cleaning" is S.

- **TBL-005 (M) Move table** (blueprint): moves an open session and all its orders from one table to a free table.
 - Kitchen tickets update **in place** with a note such as "Moved from T4 to T7". No duplicate tickets.
 - The old table's tablet resets; the new table's tablet unlocks.
- **TBL-006 (S)** Merge tables into one bill; transfer individual items between tables.
- **TBL-007 (M) Table overview** on the POS and manager dashboard, updated live without refresh (blueprint). For each table it shows:
 - state, time seated, amount spent so far, responsible waiter;
 - pending approvals and active service requests.

Tapping a table opens its current order and running total (blueprint). From there the cashier can print the bill and close the table.
- **TBL-008 (M) Takeaway orders** are created at the POS (waiter app is S) with a token number and optional customer name/phone. No table is involved.

6.5 Order management, all channels (ORD)

- **ORD-001 (M)** Order sources: POS, waiter app, table tablet, QR menu. Each order records its source, device and creating user (for staff) or table (for customers).
- **ORD-002 (M)** Orders and order items follow the state machine in Appendix B.
- **ORD-003 (M) Customer-originated orders (tablet and QR) require approval** by a waiter, or by a cashier/manager, before reaching the kitchen.
 - The approver can change quantities or remove items (with the diner's agreement) before approving, or reject with a reason.
 - The diner's screen shows the status throughout.
- **ORD-004 (M)** Approval can be given from:
 - the waiter app;
 - the POS;
 - **the table tablet itself**, by the waiter entering their PIN ("Approve at table"), so approval happens while talking to the diner.
- **ORD-005 (M)** A pending approval alerts the responsible waiter (pager + app) and escalates to the manager if not acknowledged within N seconds ⚙ (NTF-005).

- **ORD-006 (M)** Staff-originated orders (POS, waiter app) go to the kitchen directly when "Send KOT" is pressed.
- **ORD-007 (M)** When an order is approved or sent:
 - items are split by kitchen station, and one KOT is produced per station (screen, print or both per station setting);
 - combos are exploded into their components;
 - variants, modifiers and instructions appear under each item.
- **ORD-008 (M)** KOT numbers are sequential per business day. Each KOT shows table/token, waiter, source, time, KOT number and order number.
- **ORD-009 (M)** A table session can contain multiple orders and KOTs (rounds). The bill aggregates all of them.
- **ORD-010 (M)** Item status (Pending approval → Sent → Preparing → Ready → Picked up → Served, or Rejected / Cancelled / Voided) is broadcast in real time to the POS, manager dashboard, waiter app, KDS and the table's tablet (blueprint).
- **ORD-011 (M) Cancellation rules.**
 - Before submission, the diner or waiter can freely edit the cart.
 - After submission but before approval, the approver can edit or reject.
 - After *Sent* but before *Preparing*, a waiter can cancel with a reason. The KDS shows the cancellation prominently, and a "CANCELLED" slip prints if the station prints.
 - After *Preparing*, only a **void** is possible, needing a manager PIN and a reason. It appears in the voids/wastage report.
- **ORD-012 (M)** Changes after a KOT (quantity, instruction) generate a "MODIFIED" delta ticket. Nothing changes silently.
- **ORD-013 (M) Idempotency** (blueprint). Every submission carries a client-generated unique ID. Retries, double taps and relay replays **never** create duplicate orders, KOTs or stock deductions.
- **ORD-014 (M)** The server **recomputes all prices, taxes, discounts and availability** from its own data. Prices sent by any client are ignored.
- **ORD-015 (M)** Special instructions per item (≤ 140 characters ☺) and per order.
- **ORD-016 (S)** Hold and fire: a waiter can hold items (e.g., mains) and release them to the kitchen later.
- **ORD-017 (M)** If an item becomes unavailable between adding it to the cart and submission, the server rejects that line with a clear message. The user can confirm submission of the rest.

6.6 Kitchen Display System and kitchen printing (KDS)

- **KDS-001 (M)** The KDS is a browser-based mode of the web console, served by the local server. It runs full-screen on any modern browser device (Android tablet/TV, mini-PC) that has been paired as a KDS device.
- **KDS-002 (M)** Each KDS device is assigned one or more stations and shows only their tickets. An "Expo/Pass" view of all stations is S.
- **KDS-003 (M)** A ticket card shows:
 - KOT number, table/token, waiter, source, live time since sent;
 - items with quantity, variant, modifiers and instructions, with combo components grouped under the combo name;
 - badges for Moved, Modified and Cancelled.
- **KDS-004 (M)** Ticket age uses colour thresholds ⚙ (default green < 10 min, amber 10-20 min, red > 20 min), always paired with an icon or text, not colour alone.
- **KDS-005 (M)** Per-item and per-ticket actions: **Mark preparing** and **Mark ready** (blueprint). "Ready" triggers the notification to the responsible waiter (NTF). Bump removes a completed ticket; Recall restores recently bumped tickets.
- **KDS-006 (M) Ready-not-collected** (blueprint): if ready items are not picked up within X minutes ⚙ (default 3), the ticket flashes and shows a **"Notify manager"** button that sends an alert to the manager on duty. Automatic escalation without pressing the button is S ⚙.
- **KDS-007 (M)** Items are marked *picked up* at the pass on the KDS, or by the waiter in the app.
- **KDS-008 (M) Printing** (blueprint):
 - Each station's mode is screen, print or both ⚙.
 - Printers can be network (IP) or USB, ESC/POS, 80 mm or 58 mm. KOTs can be reprinted.
 - If a printer is offline, the POS and manager are alerted. In print-only mode KOTs are queued and printed on recovery, or reprinted on another printer chosen by the manager.
- **KDS-009 (M)** Audible chime for new tickets; a distinct sound for cancellations and modifications; volume ⚙.
- **KDS-010 (S)** All-day summary: total quantity of each item pending across open tickets.
- **KDS-011 (M)** Dark theme by default, readable from 2 m, touch targets ≥ 48 px, usable with wet or gloved hands.

- **KDS-012 (M)** On connection loss the KDS shows a full-screen "Disconnected" banner, reconnects automatically and resynchronises without losing or duplicating tickets.
- **KDS-013 (C)** Bump-bar hardware support.

6.7 POS billing, payments and day-end (BILL)

- **BILL-001 (M)** A bill for a table session or takeaway order shows:
 - itemised lines (variants, modifiers, combos), quantity, unit price, line total;
 - subtotal, discounts, service charge (if enabled);
 - taxes per tax group with the CGST/SGST split;
 - round-off and grand total; amount in words is S.
- **BILL-002 (M)** Bills carry GST invoice particulars, with templates configurable to the restaurant CA's guidance:
 - legal name, address, GSTIN, **FSSAI license number**;
 - invoice number and date/time, SAC code, place of supply (state), tax breakdown;
 - customer name and GSTIN, when given (B2B).
- **BILL-003 (M) Invoice numbering.** Unique and consecutive within each financial year, ≤ 16 characters, with a configurable prefix. Numbers are never reused or skipped. Cancelled invoices stay in the series with status *Cancelled*.
- **BILL-004 (M)** Tax rates are configured per tax group and never hard-coded. Prices can be tax-inclusive or tax-exclusive ⚙.
- **BILL-005 (M) Discounts:** item-level or bill-level, percentage or flat, with a mandatory reason.
 - Each role has a limit ⚙ (e.g., cashier ≤ 10 %); above the limit needs a manager PIN.
 - Complimentary (100 %) items need a manager PIN.
- **BILL-006 (M) Service charge** is disabled by default. If enabled, it is labelled voluntary on the bill and can be removed at the diner's request (audited), consistent with consumer-protection guidelines (§ 12).
- **BILL-007 (M) Split bill** by items or into equal parts. Each part gets its own invoice number.
- **BILL-008 (M) Payment recording** (no integration):
 - Modes: Cash, Card, UPI, Other ⚙, with split across modes allowed.
 - Cash tendered and change due are recorded; a reference number for card/UPI is optional.

- A bill is settled when recorded payments equal the total.
- **BILL-009 (M)** Reprints are marked "DUPLICATE" and audited.
- **BILL-010 (M) Editing a printed bill** (items, discount) needs a manager PIN and a reason, and the audit log stores before/after values. A settled bill cannot be edited: it can only be voided and re-issued, which needs a manager PIN. The voided invoice keeps its number and the new invoice gets a new number.
- **BILL-011 (M)** Customer details are optional: name, phone (with an explicit consent checkbox for digital bill/feedback), GSTIN.
- **BILL-012 (S) Digital bill** by WhatsApp or SMS through a vendor-configured provider, sent only with consent and queued while offline. Bill content leaves the restaurant only for this purpose, and any hosted copy expires ⚙ (default 30 days).
- **BILL-013 (M) Shift and day-end.**
 - Open a shift with an opening cash float; record cash in/out with a reason.
 - Close the shift with expected vs. counted cash (denomination count is S); the variance is recorded.
 - Day-end produces the Z-report and closes the business date. Open tables block day-end unless a manager carries them forward with a PIN.
- **BILL-014 (M)** Bill printing on an ESC/POS printer (80 mm/58 mm) with a configurable template (logo, header, footer) and on-screen preview.
- **BILL-015 (M)** A bill request from the tablet or waiter notifies the cashier (POS) and the responsible waiter, and sets the table state to *Bill requested*.
- **BILL-016 (S)** Cash-drawer kick through the receipt printer. "No-sale" drawer openings are audited.
- **BILL-017 (C)** Dynamic UPI QR with the amount on the bill; card-terminal and payment-gateway integration.

6.8 Waiter (Captain) app (WTR)

- **WTR-001 (M)** An Android app delivered as a signed APK, installed and updated through MDM or an in-app updater (UPD-001). It runs only on paired devices, with PIN login.
- **WTR-002 (M) Home:** "My tables" (assigned section) showing table state, seated time, pending-approval badge and active service requests. A toggle shows all tables.
- **WTR-003 (M) Order taking:**
 - open a table (covers, waiter);
 - browse categories, text search, item details;

- variant/modifier popup (shared component, MENU-012), combos, quantity, special instructions;
- **Send KOT.**
- **WTR-004 (M) Approvals inbox:** tablet and QR orders for the waiter's tables. The waiter can view, edit, approve, or reject with a reason.
- **WTR-005 (M) Service-request inbox:** Water / Waiter / Bill requests with table and age. Actions: acknowledge, resolve (clears the request, same as Cancel on the tablet).
 - Alerts use a banner, sound and vibration that are hard to miss even when the app is in the background or the screen is locked (Android foreground service, high-priority notification) (blueprint).
- **WTR-006 (M)** The app receives every alert the waiter's pager receives (order ready, new order to approve, manager nudges), and acknowledging in either place acknowledges both.
- **WTR-007 (M)** Live item status per table; mark picked up / served.
- **WTR-008 (M)** Move table (TBL-005) (blueprint); request the bill, which notifies the cashier. Printing the bill to the POS printer from the app is S.
- **WTR-009 (M)** Cancellations and voids follow ORD-011. When needed, a manager enters their PIN on the waiter's device.
- **WTR-010 (M)** Out-of-stock items are shown as unavailable in real time (blueprint).
- **WTR-011 (S)** Upsell prompts: the top recommendations (REC) for the table's current order.
- **WTR-012 (M) Connectivity:**
 - A persistent "Offline" indicator shows when the server can't be reached.
 - Unsent orders are kept as drafts on the device and resent on reconnection with the **same idempotency key**. They are never silently discarded.
 - The waiter always sees clearly whether each KOT reached the kitchen.
- **WTR-013 (S)** Create takeaway orders.
- **WTR-014 (M)** Shows the waiter's own pager with battery and connection status, and warns when the pager is offline or low on battery.

6.9 Customer table tablet (TAB)

- **TAB-001 (M)** Android app locked in **kiosk mode** (Android device-owner lock-task mode, managed by MDM). Diners cannot leave the app, change settings or open other apps. Staff exit through a hidden gesture plus a manager PIN.

- **TAB-002 (M)** Each tablet is paired to one table (AUTH-009).
- **TAB-003 (M) Session lifecycle.**
 - While the table is not open, the tablet shows an idle screen (restaurant branding, promotions 🌀, "Please wait to be seated").
 - It unlocks when a table session is opened (TBL-003).
 - When the session closes, the cart, order history, feedback and any customer data on the device are wiped.
- **TAB-004 (M) Home screen with four large labelled buttons: Water, Waiter, Bill, Cancel** (blueprint), plus the entry to the menu.
 - **Water / Waiter / Bill** each raise a *service request* to the table's responsible waiter (pager + app). **Bill** also notifies the cashier (BILL-015).
 - A raised request **stays active**, with the button showing "Requested, waiter notified" and the elapsed time, until it is cancelled or resolved.
 - **Cancel works like a flight-cabin call button.** It cancels the table's active request(s):
 - The waiter presses it on arrival (resolving the request).
 - The diner presses it if the request is no longer needed.
 - If more than one request is active, the tablet asks which to cancel. With one active request, Cancel clears it directly.
 - While active, the request re-alerts the waiter every R seconds 🌀 until acknowledged. If it is not acknowledged within N seconds 🌀 it escalates directly to the manager (NTF-005). After acknowledgement the tablet shows "Waiter is on the way".
 - The same request type cannot be raised again while one is active (anti-spam).
- **TAB-005 (M) Menu browsing:**
 - categories, and item cards with photo, price, veg/non-veg/egg marker, spice level, tags and bestseller badge;
 - item detail with description;
 - filters (veg only, Jain, spicy, bestseller) and sorting;
 - a note on whether prices include or exclude taxes.
- **TAB-006 (M) Search:** fuzzy text matching over names, synonyms (MENU-011) **and tags**. So "something spicy" returns items tagged spicy, and "paneer dishes" returns items whose name, synonyms or tags mention paneer (blueprint, extended).
- **TAB-007 (S) Voice search** (blueprint): a microphone button uses Android's on-device speech recognition (English) and feeds the recognised text into TAB-006.

- It works offline **only** if the tablet has Android's offline English speech pack installed, which is a site-readiness checklist item (ONB-011).
- If recognition isn't available, the button explains this and the diner can type instead.
- **TAB-008 (M) Cart:** variant/modifier popup (shared component), quantity, special instructions, and a review screen with the subtotal and estimated taxes.
- **TAB-009 (M) Submit** → status "Waiting for waiter confirmation" and the waiter is notified (ORD-003, ORD-005). After approval, each item's status is shown live: Received → Preparing → Ready → Served (blueprint note).
- **TAB-010 (M) My order:** all items ordered in this session, their status and the running total.
- **TAB-011 (M) Recommendations** (REC) on the menu, item and cart screens, each with its reason (e.g., "Goes well with Butter Chicken", "Bestseller").
- **TAB-012 (M)** Out-of-stock items become unavailable in real time (blueprint).
- **TAB-013 (M)** If the tablet can't reach the server, ordering and service buttons are disabled with the message "Please call a waiter", and the tablet recovers automatically.
- **TAB-014 (S) Feedback after the bill:** food and service ratings (1-5) plus an optional comment. The diner can optionally give a phone number, with explicit consent, for a digital bill and feedback follow-up.
- **TAB-015 (M) Device health** (battery %, charging, connectivity, app version) is reported to the manager dashboard. A low-battery alert fires at 20 % 🔌.
- **TAB-016 (M) Diner-friendly design:** large text, high contrast, plain language, no login, and first order placed unaided within 2 minutes (§ 7.4).
- **TAB-017 (S)** Estimated preparation time per order, from item prep times and current kitchen load.

6.10 QR online menu and dine-in ordering (QR)

- **QR-001 (M)** Each table has a unique QR code (printable table tent) encoding the restaurant, the table and an unguessable token. Regenerating a table's code invalidates the old one.
- **QR-002 (M)** A **public, mobile-first website** (no app, no login) shows the restaurant's live menu: items, photos, prices, tags, availability, variants, modifiers and combos. It uses the same data model and selection behaviour as every other surface (MENU-012) (blueprint).
- **QR-003 (M) Menu sync** (blueprint): menu, price and availability changes are pushed from the local server to the cloud within 30 s when online. While offline, changes are queued in

a local outbox and retried automatically with backoff, never silently dropped.

- **QR-004 (M)** Browse, search (with synonyms and tags), filter, cart, special instructions and recommendations: best sellers and rules, plus learned pairings from the nightly snapshot.
- **QR-005 (M) Dine-in only. No payment before ordering.** Customer name is optional.
- **QR-006 (M)** Orders are submitted to the relay **through a server-side function**, never by writing to the database directly from the browser. The function:
 - validates the table token;
 - validates items against the latest menu snapshot;
 - enforces rate limits (default 5 submissions per 10 minutes per table and per browser ⚙️);
 - stores the order with its idempotency key.
- **QR-007 (M) Customer-visible status** (blueprint):
 1. "Sending your order to the restaurant..."
 2. "Received, waiting for waiter confirmation"
 3. "Your order has been sent to the kitchen"
 4. Item-level statuses (S).

If the restaurant has not picked up the order within **T seconds ⚙️ (default 60)**, the page shows "We couldn't reach the restaurant right now. Please call a waiter." The relay marks the order *expired*, and the local server rejects expired orders if they arrive later. This way no diner waits for an order that was never received.

- **QR-008 (M)** The local server sends a presence signal to the relay every 30 s ⚙️. If none arrives for 2 minutes ⚙️, the QR site switches to **read-only menu mode** with a banner and ordering disabled.
- **QR-009 (M)** Once received locally, a QR order becomes a normal order (source = QR) that needs waiter approval (ORD-003). The rest of the system treats it like any other order (blueprint).
- **QR-010 (M)** If a QR order arrives for a table that isn't open, it still enters *pending approval*, routed to the responsible waiter of that table's section. Approving it opens the table.
- **QR-011 (S)** Call waiter / request bill from the QR page, using the same service-request flow as the tablet, for tables without tablets.

- **QR-012 (M)** Performance: the first menu view's Largest Contentful Paint is ≤ 2.5 s on a mid-range Android phone over 4G. Photos are served through a CDN in responsive sizes.
- **QR-013 (M)** A privacy notice and terms are linked on the page. No advertising or third-party tracking; only essential, anonymous usage metrics.
- **QR-014 (M) Tenant isolation.** No QR code, token or credential of one restaurant can read or write another restaurant's menu or orders. This is verified by automated tests (SEC-005) (blueprint: dedicated isolation test).

6.11 Recommendation engine (REC)

- **REC-001 (M)** A three-layer engine runs inside the local server. Results are served to the tablet and waiter app, and published as a snapshot to the QR relay.
- **REC-002 (M) Layer 1, manual rules (highest priority).** The restaurant defines rules such as "if the order contains item/category X, suggest item/category Y", each with a priority, an optional time-of-day window, an optional date range and target channels. Rules are managed on the dashboard.
- **REC-003 (M) Layer 2, learned "frequently ordered together" (machine learning).**
 - Association-rule mining (e.g., FP-Growth/Apriori, or co-occurrence with lift) over **that restaurant's own orders** in a rolling window (90 days ⚙️).
 - Retrained nightly on the local server; no cloud required.
 - Minimum support, confidence and lift thresholds ⚙️.
 - Activated only after a minimum order volume (300 orders ⚙️) to avoid noisy suggestions.
- **REC-004 (M) Layer 3, best sellers.** Used when nothing has been ordered yet or no rule applies. Ranked by quantity over the last 30 days ⚙️ and segmented by time of day (breakfast / lunch / evening / dinner windows ⚙️).
- **REC-005 (M)** Filters applied to all layers:
 - exclude out-of-stock items;
 - exclude items already in the order, unless the item is marked "repeatable" (e.g., beverages, breads) ⚙️;
 - respect the table's veg-only filter;
 - **course awareness:** favour the next course in a configurable category sequence (starters → mains → breads → desserts → beverages ⚙️).
- **REC-006 (M)** Every recommendation carries a human-readable reason label.

- **REC-007 (M)** Recommendations are **table-level only**: no personal profiles and no cross-visit history in v1.
- **REC-008 (M)** Tracking per layer and per item: impressions, taps, add-to-cart, ordered. This feeds the recommendation-performance report (RPT-012).
- **REC-009 (S)** The manager can boost or pin items (e.g., high margin, overstocked) and block items from being recommended.
- **REC-010 (C)** Personalised recommendations with the diner's consent; A/B testing of strategies.
- **REC-011 (M)** Recommendations are returned in ≤ 200 ms on the LAN.

6.12 Notifications, alerts and escalation (NTF)

- **NTF-001 (M)** A central notification service in the local server turns **events** into **alerts** using **rules** that decide **recipients** and **channels**. Channels: pager, waiter app, POS, manager dashboard, KDS, table tablet display.
- **NTF-002 (M)** The manager can configure rules on the dashboard (brief: "notifications as per the configurations made"). For each event type:
 - recipients: responsible waiter, all waiters in the section, all waiters, manager(s) on duty, cashier, or a station;
 - channels;
 - escalation timeout **N** ⚙ and repeat interval **R** ⚙.

Factory defaults are in Appendix C.

- **NTF-003 (M)** Event types (minimum):
 - customer order pending approval; item/ticket ready (blueprint);
 - service request Water / Waiter / Bill (blueprint);
 - ready-not-collected, via the kitchen's "Notify manager" button (blueprint);
 - **manager nudge to a waiter** (blueprint);
 - order cancelled/modified (to kitchen);
 - device offline or low battery (pager, tablet, KDS, printer);
 - disk space, backup failure, license state change, update available/installed.
- **NTF-004 (M) Acknowledgement.** The recipient acknowledges on the pager button or in the app. Acknowledging stops repeats and escalation; the acknowledgement time is recorded for reports.

- **NTF-005 (M) Escalation.** If an alert is not acknowledged within **N seconds** ⚙️ (**default 60**), it goes **directly to the manager(s) on duty** (on the POS, the manager dashboard, and the manager's pager/app if one is assigned). Escalations are logged and appear in staff reports.
- **NTF-006 (M) Delivery.** Alerts are delivered at least once and de-duplicated on the device. Devices resynchronise missed alerts on reconnection. Latency is ≤ 2 s (95th percentile) from event to pager vibration on the LAN.
- **NTF-007 (M)** If the responsible waiter's pager **and** app are both offline, the alert goes to the manager immediately, without waiting N seconds.
- **NTF-008 (M) Manager nudge:** choose one or more waiters, then a preset message (e.g., "Come to counter", "Check T7") or short text of ≤ 40 characters so it fits the pager display.
- **NTF-009 (S)** A waiter can set "On break". Their alerts then go to the manager until they return.

6.13 Wrist pager (PGR)

Hardware

- **PGR-001 (M)** A wrist-worn device based on the ESP32 family (ESP32-S3 or equivalent), using a **pre-certified radio module**, with:
 - a vibration motor and an LED;
 - a display showing at least 2 lines $\times$ 12 characters, legible indoors (e.g., "T7 READY");
 - at least one physical button (acknowledge);
 - a rechargeable battery with USB-C or dock charging.
- **PGR-002 (M) Battery life ≥ 14 hours** of shift use at a typical load (≤ 60 alerts per hour). This is a **prototype gate**: it must be demonstrated on real hardware before pager development goes beyond the prototype (§ 17).
- **PGR-003 (S)** Splash resistance (target IP54), a comfortable strap, and operation at 0-45 °C (kitchen environment).
- **PGR-004 (M)** The pilot may use off-the-shelf ESP32 watch hardware (for example LilyGO T-Watch S3) if it meets PGR-001 and PGR-002. Custom hardware is future scope.

Firmware

- **PGR-005 (M)** Connects to the staff Wi-Fi (WPA2/WPA3) and to the local MQTT broker **over TLS with a per-device credential**. The pager subscribes only to its own topic, and broker

access rules stop any pager reading another's messages.

- **PGR-006 (M) On an alert** the pager:
 - vibrates with a pattern per alert type ⚙ (e.g., ready = one long, service request = two short, manager/escalation = three);
 - lights the LED and shows short text ("T7 READY", "T4 WATER", "T5 NEW ORDER", "MGR: COME TO COUNTER").

Several alerts are queued and a count is shown. A short press acknowledges the current alert; a long press scrolls through queued alerts.

- **PGR-007 (M) Heartbeat** every 30 s ⚙ with battery %, signal strength (RSSI) and firmware version. The server marks a pager offline after 3 missed heartbeats. The pager shows a "no connection" icon and vibrates once when it loses the connection.
- **PGR-008 (M) Reliable delivery:** MQTT QoS 1 with message IDs for de-duplication. Unacknowledged alerts are delivered again after reconnection.
- **PGR-009 (M) Power management** (Wi-Fi power save / light sleep between messages) sufficient to meet PGR-002 without breaking NTF-006 latency.
- **PGR-010 (M) Production devices enable **secure boot and flash encryption**.** Credentials live in encrypted storage. The debug/serial console is disabled or locked. There are no default passwords.
- **PGR-011 (M) Over-the-air firmware updates** are signed, use A/B partitions and roll back automatically if the new firmware fails to boot. They are distributed through the local server from firmware released in the Vendor Control Plane.
- **PGR-012 (M) Pairing and assignment:** the manager pairs a pager (scan its QR code/serial) and assigns it to a waiter. The server stores which pager belongs to which waiter (blueprint). Re-assignment at shift start takes ≤ 30 s.
- **PGR-013 (M) At low battery (≤ 15 % ⚙)** the pager alerts both the wearer and the manager.
- **PGR-014 (M)** A manager can also wear a pager to receive escalations.

6.14 Manager dashboard (MGR)

- **MGR-001 (M)** A web app served by the local server, from the same codebase as the POS. It opens on the restaurant PC and on any **paired** browser device (phone, laptop) on the staff network, for Manager and Owner roles only (blueprint).

- **MGR-002 (M) Live table overview** (TBL-007) (blueprint).
- **MGR-003 (M) Live order feed** across the restaurant with kitchen status, filterable by station, waiter and source, with delayed items highlighted (blueprint).
- **MGR-004 (M) Staff management** (blueprint):
 - add, edit and deactivate staff (never delete, to keep audit history);
 - roles; set/reset PINs;
 - section assignment per shift; pager assignment.
- **MGR-005 (M) Menu management** (MENU), including photos, descriptions, variants, modifiers, combos, availability and stock counts (blueprint).
- **MGR-006 (M) Device management:** every paired device with type, assigned table/station/person, status, battery, app/firmware version and last seen. Actions: pair, unpair, rename. "Locate" (make a tablet beep or a pager vibrate) is S.
- **MGR-007 (M) Configuration:** notification rules, recommendation rules, stations and printers, service charge, discount limits, and all ⚙️ timers and thresholds in this BRD. Tax and invoice settings are Owner only.
- **MGR-008 (M) Alert centre:** escalations, kitchen flags, device and system alerts, each with acknowledge.
- **MGR-009 (M) Reports and exports** (RPT); audit log viewer (AUD).
- **MGR-010 (M) System:** storage usage, backup status, archive and purge (Owner, DATA), license status and reminders (LIC), update status and "install now" approval (UPD), "send diagnostics to support".
- **MGR-011 (M)** Responsive layout, usable on a 360 px-wide phone and on desktop.

6.15 Reports and exports (RPT)

All reports can be filtered by business-date range, and where relevant by waiter, station, source, order type and payment mode.

- **RPT-001 (M) Sales summary** by day, by hour and by business-date range (blueprint).
- **RPT-002 (M) Item-wise and category-wise sales:** quantity, gross, discounts, net, tax (blueprint: by menu item).
- **RPT-003 (M) Waiter-wise** (blueprint):
 - orders created and approved, sales;
 - average acknowledgement time for service requests and alerts;
 - escalations, cancellation/void requests.

- **RPT-004 (M) Source-wise** (POS / waiter / tablet / QR) and **order-type** (dine-in / takeaway) sales.
- **RPT-005 (M) Payment-mode report, shift report** with cash variance, and **Z-report** (day-end).
- **RPT-006 (M) GST reports:**
 - tax summary: taxable value, CGST and SGST by rate and SAC;
 - **invoice register:** every invoice number in sequence, including cancelled ones.

Both are made for the restaurant's CA to file returns.

- **RPT-007 (M) Discounts and complimentary items:** who requested, who approved, the reason and the value.
- **RPT-008 (M) Cancellations and voids:** item or bill, the stage at which it happened, who, approver, reason and value (wastage).
- **RPT-009 (M) Reprints and post-print bill edits.**
- **RPT-010 (M) Kitchen performance:** average prep time by item and station, ready-to-pickup time, "Notify manager" events.
- **RPT-011 (M) Table metrics:** covers, table turnover, average seated duration, average spend per cover.
- **RPT-012 (M) Recommendation performance:** attach rate and revenue from recommended items, per layer (REC-008).
- **RPT-013 (S) Customer feedback:** ratings trend and comments.
- **RPT-014 (M) Suspicious activity report** (AUD-006).
- **RPT-015 (M) Audit details** (brief: "which waiter generated the order etc."). Order-level report rows and drill-downs show:
 - created by, source device, approved by;
 - KOT times and each status timestamp;
 - modified/cancelled/voided by, and the bill printed/settled by.
- **RPT-016 (M) Dashboard charts** for key metrics (blueprint), with table views and drill-down to individual orders and bills.

- **RPT-017 (M) Export to PDF, Excel (.xlsx) and CSV.** Every export is stamped with the restaurant, the filters, "generated by" and "generated at". Exports are audited. This is how the owner receives reports remotely: staff export and send them (decision Q2).
- **RPT-018 (M)** Any report over 12 months of data at design capacity (§ 7.2) is generated in ≤ 5 s.
- **RPT-019 (C)** Scheduled daily summary e-mailed to the owner.

6.16 Audit trail and anti-fraud (AUD)

- **AUD-001 (M)** An **append-only audit log** records every security-relevant and money-relevant action:
 - **Access:** login, logout, failures, lockouts, overrides.
 - **Orders:** create, approve, reject, modify, cancel, void.
 - **Money:** discounts, complimentary items, service-charge removal; bill print, reprint, edit, void; payments; cash in/out; shift close; day-end.
 - **Menu:** price, menu and stock changes.
 - **Administration:** staff, role and permission changes; device pairing; settings changes.
 - **System:** exports, backup, restore, archive, purge; license and update events.
- **AUD-002 (M)** Each entry records:
 - server timestamp and business date;
 - actor, approver (for overrides) and device;
 - action, entity, before/after values and reason.
- **AUD-003 (M) Tamper-evident:** entries are hash-chained (each includes the hash of the previous one). The current chain-head hash is sent to the Vendor Control Plane with each heartbeat, so later tampering with the local database can be detected. A verification tool is available on the dashboard.
- **AUD-004 (M)** Orders, bills, payments and audit records can **never be hard-deleted** through the application. The database role used by the application has no DELETE permission on these tables. Only the archive-and-purge process (DATA-008) can remove them, and it writes a purge record with checksums.
- **AUD-005 (M)** Audit viewer with filters and export.
- **AUD-006 (M) Daily suspicious-activity report** 🌀, flagging for example:
 - voids after the bill was printed; bill edits after printing;
 - discounts above threshold; repeated reprints;

- cash variance above threshold;
 - items cancelled after preparation started;
 - **items sent to the kitchen that never appeared on a settled bill;**
 - "no-sale" cash-drawer openings.
- **AUD-007 (M)** Audit records are retained like financial records (§ 9.3).

6.17 Storage, backup, archive and purge (DATA)

- **DATA-001 (M)** The PostgreSQL data directory and photo store sit on the drive chosen at installation (ONB-002). PostgreSQL listens on localhost only.
- **DATA-002 (M) Automatic local backups:**
 - continuous or hourly ⚙ incremental backup (WAL archiving);
 - a nightly full backup to a **second location** (second disk, external USB drive or NAS ⚙).

Every backup is integrity-checked. Default retention: 14 daily + 8 weekly ⚙.

- **DATA-003 (M) Encrypted cloud backup:** a nightly backup, encrypted (AES-256) on the restaurant PC before upload, stored in India.
 - The encryption key is generated at installation and stored locally (DPAPI).
 - A copy is escrowed in the Vendor Control Plane, wrapped by a vendor KMS key and released only on the Owner's authenticated restore request (audited). The policy is open item OI-10.
 - On by default. The Owner may opt out (recorded in the audit log).
- **DATA-004 (M) Restore:** one-click restore from a local or cloud backup onto the same or a new PC (after license re-binding, LIC-004).
 - Targets: RTO ≤ 2 hours once replacement hardware is available.
 - RPO ≤ 1 hour when local backups survive; ≤ 24 hours when only the cloud backup survives.
 - The restore procedure is documented and rehearsed before the pilot.
- **DATA-005 (M)** Backup failures alert the manager and are reported to the Vendor Control Plane.
- **DATA-006 (M) Storage monitoring:** the dashboard shows disk usage by category (transactions, audit, logs, photos, backups). Warnings at **80 %** and **90 %** ⚙ of the drive.

- **DATA-007 (M) Operational data is purged automatically** after 90 days ⚙: notifications, device heartbeats, debug logs, temporary files, unused photo renditions.
- **DATA-008 (M) Archive-then-purge for business records** (brief: "clear some data periodically"). Business records are orders, bills, payments, audit and derived report data. Steps:
 1. The Owner (password + 2FA) selects a period (whole financial years ⚙).
 2. The system writes an **encrypted, checksummed archive**, plus a human-readable export (invoice register CSV and invoice PDFs), to a chosen location, and adds it to the cloud backup set.
 3. It **verifies the archive by reading it back**.
 4. Only then does it delete that period from the live database, and writes a purge record.

The purge screen warns clearly if the period is still within the statutory retention period (§ 12). The restaurant must keep the archive for that period.

- **DATA-009 (S) Archive viewer:** open an archive read-only to view or print historical invoices and reports without restoring it into the live database.
- **DATA-010 (M) Full data export** in open formats (CSV/JSON plus invoice PDFs) is available to the Owner **at any time, including in restricted license mode** (LIC-005).
- **DATA-011 (M) Personal-data erasure:** on a diner's request, their phone number/name is erased or anonymised in non-invoice records. Invoices are kept as tax law requires.
- **DATA-012 (M) Sizing target:** 5 years of transactions for a restaurant at design capacity (§ 7.2), excluding photos and backups, fit in ≤ 100 GB. The development team validates this with a data-volume test.

6.18 Licensing and subscription enforcement (LIC)

- **LIC-001 (M)** Each installation holds a **license file signed by the vendor** (Ed25519 or equivalent) containing:
 - restaurant ID, installation ID and PC fingerprint;
 - plan, entitled modules (all modules in v1), device limits (tablets, pagers, waiter devices, KDS screens);
 - issue date, *valid-until* date, grace days and state.
- **LIC-002 (M) Rolling validity:** a license is valid for **30 days** ⚙ from issue and renewed automatically (at least daily) while online. The restaurant works **fully offline** until valid-

until (blueprint: cached license).

- **LIC-003 (M)** The local server verifies the license at start-up and periodically, and client apps get their entitlements from the server. The vendor's public key is embedded in the software; a missing, unsigned or tampered file puts the installation into *Restricted* state.
- **LIC-004 (M) Machine binding:** the license is bound to a PC fingerprint that tolerates minor hardware changes (e.g., 3 of 5 identifiers must match). Moving to a new PC is a support action in the Vendor Control Plane (audited).
- **LIC-005 (M) License states** (brief: "remotely stop its functioning if payment is not made"):

State	Entered when	Restaurant experience
Active	Subscription paid	Full functionality
Payment due	Invoice overdue (recorded in Control Plane)	Daily polite reminder on POS and dashboard; no restriction
Grace	Vendor moves to grace, or overdue beyond N days ⚙	Banner on all staff screens with days remaining (7 days ⚙)
Restricted	Grace ends, license expires while offline, license invalid/tampered, or vendor suspension	No new table sessions, orders or KOTs. Can still: finish and bill already open tables, print, view reports, export all data . Backups continue.
Terminated	Contract ended	As Restricted; data export stays available; nothing deleted

Rules that always apply:

- A move into Restricted takes effect **only at the start of the next business day**, never in the middle of service.
- The platform **never deletes, encrypts or withholds** the restaurant's data.
- All state changes are logged in the Control Plane and locally.
- **LIC-006 (M) Reactivation** takes effect within 1 minute of payment being recorded in the Control Plane (when online). For offline sites, a signed reactivation code can be entered by the Owner (e.g., received by phone).
- **LIC-007 (M) Anti-tamper** measures:

- clock-rollback detection (last-seen time stored and signed, compared with the server time in heartbeats);
- signed binaries, integrity checks and obfuscation of the license module (SEC-011).

Tamper events are reported to the Control Plane.

- **LIC-008 (M)** The license state and remaining days are always visible to the Owner. Reminder wording is polite and precise.
- **LIC-009 (M)** Timelines and behaviours in LIC-005 must match the subscription agreement (Business Owner, § 19).
- **LIC-010 (C)** Selling modules separately, using the entitlement fields already in the license.

6.19 Remote updates and support (UPD)

- **UPD-001 (M)** All components can be updated remotely (blueprint):
 - Windows app + local server + web console: signed installer or delta update.
 - Waiter and tablet apps: over-the-air JavaScript updates (e.g., Expo EAS Update); native changes through a new APK pushed by MDM or the in-app updater.
 - Pager firmware: OTA via the local server (PGR-011).
- **UPD-002 (M) Release channels:** Stable and Pilot/Beta, plus **per-restaurant pinning**, managed in the Vendor Control Plane. A single restaurant can be given a hotfix build or held on a specific version (brief: "code fixes remotely for a specific restaurant").
- **UPD-003 (M) Staged rollout** by percentage or cohort, with pause and halt. Rollout halts automatically if error rates or heartbeat failures spike after an update.
- **UPD-004 (M) Installation timing:** updates download in the background but install only inside a **maintenance window** 🕒 (**default 03:00-06:00**), or when a manager presses "Install now". Never while tables are open, unless the update is flagged security-critical **and** a manager approves.
- **UPD-005 (M) Safe updates:**
 - a full database backup and integrity check before every update;
 - forward-only migrations, tested against production-like data;
 - a health check after the update;
 - automatic rollback (application and database) if the health check fails.
- **UPD-006 (M) Compatibility:** the server supports the current and previous minor version of its client API (N-1), so tablets, phones and pagers that update later keep working. A component version matrix is published with every release.

- **UPD-007 (M)** Every update artefact is **signed** (Authenticode for Windows, APK signature, EAS Update code signing, firmware signature) and verified before installation. Downloads are over HTTPS only.
- **UPD-008 (M) Remote diagnostics:** the manager can "Send diagnostics to support" (logs, versions, configuration without secrets, device list). Support can request diagnostics from the Control Plane, which prompts the manager to allow it.
- **UPD-009 (M) No remote shell or hidden access.** Remote screen sharing is only possible through a session **started by the restaurant** with a standard tool. Every vendor support action is audited.
- **UPD-010 (M)** Vendor-controlled remote configuration (feature flags, vendor thresholds) can be pushed from the Control Plane. Such settings are visible to the manager and recorded in the audit log.

6.20 Vendor Control Plane (VCP)

- **VCP-001 (M)** A cloud web application for vendor staff with roles Super Admin, Onboarding, Support, Finance and Read-only. Multi-factor authentication is mandatory; every action is audited.
- **VCP-002 (M) Tenant management:** create a restaurant (contacts, GSTIN, plan, device limits); generate activation codes; re-bind an installation to a new PC; provision the restaurant in the QR relay with its own scoped credentials.
- **VCP-003 (M) Subscriptions and licenses:** plan catalogue, billing cycles, **manual payment recording** (v1), overdue tracking, license state changes (LIC-005) with a reason, issuing/revoking licenses, offline reactivation codes.
- **VCP-004 (S)** Automated subscription collection through a payment gateway (e.g., UPI Autopay/e-mandate), and the vendor's own GST invoices.
- **VCP-005 (M) Fleet monitoring** from heartbeats (every 5 min ⚙️). Per restaurant:
 - online/offline and last seen;
 - versions of every component, disk usage, backup status;
 - device counts and health, error counts;
 - license state, audit chain head.

Alerts fire when a restaurant is offline > 30 min during its business hours ⚙️, a backup fails, or disk usage passes 90 %.

- **VCP-006 (M) Release management:** builds come from CI already signed. Define channels, rollouts and per-restaurant pinning; halt and roll back (UPD).
- **VCP-007 (M) Support tools:** view diagnostics bundles, keep notes per restaurant, request diagnostics. There is no access to a restaurant's business data except through diagnostics or backups, under a documented access policy.
- **VCP-008 (M) Onboarding services:** the AI menu-import pipeline (ONB-006), activation, QR tenant provisioning.
- **VCP-009 (M)** Hosted in an India region, with separate development, staging and production environments.
- **VCP-010 (C)** Reseller/partner accounts.

6.21 Modularity and integration readiness (INT)

Decision Q15: v1 is sold only as a complete ecosystem, but the code must be separated so that tablet, pager and QR services can later work with other POS systems.

- **INT-001 (M)** The platform is divided into **modules with explicit boundaries**:
 - Core POS (menu, tables, orders, billing);
 - Notifications and Pager;
 - Table Tablet;
 - QR and Relay; Sync;
 - Recommendations; Reporting; Licensing.

Modules communicate **only through versioned, documented contracts** (internal APIs and domain events). A module never reads or writes another module's tables directly.

- **INT-002 (M)** Contracts are defined once in a shared package (Zod schemas) (blueprint). They are published as **OpenAPI** (REST) and **AsyncAPI** (events) documents and checked by contract tests in CI.
- **INT-003 (M) Ports and adapters.** The peripheral modules (Tablet, Pager/Notifications, QR, Recommendations) depend only on provider interfaces: *Menu Provider*, *Table Provider*, *Order Sink*, *Order Status Source*. In v1 the Core POS implements these interfaces. A future adapter for a third-party POS can implement them **without changing the peripheral modules**.
- **INT-004 (M) Domain event catalogue** (minimum):

- Menu: MenuPublished , ItemAvailabilityChanged .
 - Tables: TableOpened , TableMoved , TableClosed .
 - Orders: OrderSubmitted , OrderApproved , OrderRejected , KotCreated , ItemStatusChanged .
 - Service requests: ServiceRequestRaised , ServiceRequestAcknowledged , ServiceRequestCancelled .
 - Billing: BillRequested , BillPrinted , BillSettled .
 - System: AlertEscalated , DeviceStatusChanged .
- **INT-005 (M)** Orders carry source and externalRef ; menu items, tables and staff carry an externalId . This lets them be mapped to other POS or aggregator systems later.
 - **INT-006 (S)** A public Integration API (authenticated, rate-limited) and outbound webhooks, documented but **disabled by default** in v1.
 - **INT-007 (C)** "Module mode" installation (only some modules licensed); adapters for specific third-party POS systems; aggregator integration; accounting export.

7. Non-functional requirements

7.1 Performance

ID	Measure	Target (95th percentile unless stated)
NFR-P01	Approved/sent order → visible on KDS	≤ 1 s
NFR-P02	KOT printed after sending	≤ 3 s
NFR-P03	Kitchen "Ready" / service request → pager vibrates	≤ 2 s
NFR-P04	Availability change → all in-restaurant devices	≤ 2 s
NFR-P05	Availability or menu change → QR menu (online)	≤ 30 s

NFR-P06	QR order submitted → received by local server (online)	≤ 5 s
NFR-P07	UI response to a tap (visual feedback)	≤ 100 ms; screen transitions ≤ 300 ms
NFR-P08	Bill generation and print	≤ 3 s
NFR-P09	Recommendations returned	≤ 200 ms
NFR-P10	Report over 12 months of data	≤ 5 s
NFR-P11	Client reconnection after server restart	≤ 30 s, with full resynchronisation
NFR-P12	QR menu first view (mid-range Android, 4G)	LCP ≤ 2.5 s

7.2 Capacity (design targets per restaurant)

The system must meet § 7.1 at: **100 tables, 60 table tablets, 30 pagers, 20 waiter devices, 6 KDS screens, 8 printers, 2,000 orders per day, peak 300 orders per hour, 200 concurrent real-time connections, 1,000 menu items, 5 years of history.** These figures are validated by load tests (§ 18).

7.3 Availability, resilience and offline behaviour

- **NFR-A01 (M)** In-restaurant operations **never depend on internet access.**
- **NFR-A02 (M)** Local system availability ≥ 99.9 % during trading hours, excluding site power loss and hardware failure.
- **NFR-A03 (M)** **No committed order, KOT, bill or payment is lost** on a crash or power cut. All state changes are transactional and durable before they are acknowledged to the client.
- **NFR-A04 (M)** The watchdog restarts a failed server process within 10 s. Services start automatically at boot, before any user logs in, and the POS window reopens automatically.

Expected behaviour by failure scenario:

Failure	Expected behaviour
Internet down	Everything inside the restaurant works. QR ordering shows read-only mode (QR-008). Cloud backups, e-bills, menu sync and heartbeats queue and resume automatically. License stays valid until valid-until.
Staff Wi-Fi / LAN down	Devices show a clear offline state. Waiter-app drafts are kept (WTR-012). The POS on the server PC keeps working and can print KOTs for orders entered there.
A tablet, phone or KDS fails	Only that device is affected. Replace it and pair a new device in ≤ 5 min.
Printer fails	Alert shown. KOTs queued or redirected (KDS-008).
Pager offline or battery flat	Alerts still reach the waiter app; if both are unavailable, the manager (NTF-007).
Server process crash	Watchdog restart ≤ 10 s; clients reconnect and resync (NFR-P11).
Server PC reboot or power cut	UPS gives ≥ 15 min (§ 11). Graceful shutdown on UPS low-battery signal is S. Full recovery on power-up without manual steps.
Disk failure / PC destroyed	Restore to a new PC (DATA-004).
QR relay or vendor cloud outage	QR ordering unavailable (read-only menu or error page). No effect on in-restaurant service or the license.

7.4 Usability and user interface (Business Owner assumption B: modern, user-friendly UI)

- **NFR-U01 (M) One design system** across every app: shared tokens (colour, typography, spacing, radius, elevation), a component library for web and one for React Native with matching look and behaviour, iconography and motion guidelines. The design system is built in Phase 0 (§ 17).
- **NFR-U02 (M)** Clean, modern visual design, with light and dark themes. The KDS defaults to dark. Restaurant branding (logo, accent colour) applies to the customer-facing tablet and QR menu.

- **NFR-U03 (M)** Touch targets ≥ 48 dp. Primary flows are short:
 - calling for water: 1 tap;
 - reordering a known item and sending the KOT from the waiter app: ≤ 5 taps;
 - approving a pending order: ≤ 2 taps.
- **NFR-U04 (M)** Every screen has designed empty, loading and error states. Error messages are in plain language and say what to do next. Destructive actions ask for confirmation.
- **NFR-U05 (M)** The QR menu and manager dashboard meet **WCAG 2.2 AA** (contrast, focus, labels, scalable text). Colour is never the only carrier of meaning, on any surface.
- **NFR-U06 (M)** Usability targets, tested in the pilot with at least 5 users per role:
 - a new waiter is proficient in ≤ 30 min;
 - a first-time diner places an order on the tablet unaided in ≤ 2 min;
 - a cashier settles a split bill in ≤ 1 min.
- **NFR-U07 (S)** Contextual help and short in-app tips for first use; downloadable training videos.

7.5 Localisation

- **NFR-L01 (M)** v1 ships in **English only**.
- **NFR-L02 (M)** All UI strings are externalised in resource files, so languages can be added later without code changes.
- **NFR-L03 (M)** Formats: currency ₹ with Indian digit grouping (1,00,000.00); dates DD-MM-YYYY; 12/24-hour time ⚙; time zone Asia/Kolkata (stored as UTC).

7.6 Compatibility

Surface	Supported
Restaurant PC	Windows 11 64-bit (Pro recommended). Windows 10 reached end of support in October 2025, so it is supported only if the PC is enrolled in Microsoft Extended Security Updates.
Waiter phones and table tablets	Android 11 or later, on models from the supported-hardware list (§ 11)
KDS	Chromium-based browser (recent version) on Android tablet/TV or PC

Manager dashboard (browser)	Latest two versions of Chrome, Edge, Safari, Firefox
QR menu	iOS Safari (iOS 16+) and Android Chrome, latest two major versions

7.7 Maintainability and code quality

- **NFR-M01 (M) Object-oriented design and SOLID principles** (brief: "OOPS principles"):
 - clear domain models and services in the backend;
 - dependency injection (NestJS modules);
 - **reusable components** so the same code is used everywhere it applies (PIN pad, menu item card, modifier popup, order status chip, etc.).
- **NFR-M02 (M)** Pricing, tax, discount, order state machines and recommendation logic live in a **framework-free domain package** with unit tests. They are shared by the server and, where useful, the clients.
- **NFR-M03 (M)** TypeScript `strict` mode everywhere (blueprint). Linting and formatting are enforced in CI. `any` is not allowed without a documented reason.
- **NFR-M04 (M)** Test coverage: $\geq 85\%$ of lines in the domain package and $\geq 70\%$ overall for the backend. End-to-end tests cover every critical scenario in § 18.
- **NFR-M05 (M)** Every change goes through a pull request with at least one reviewer, and two for security-sensitive code (auth, licensing, billing, crypto, sync).
- **NFR-M06 (M)** Documentation:
 - ADRs for significant decisions;
 - generated API/event docs (INT-002);
 - runbooks for installation, restore, update, rollback and incident response;
 - an onboarding guide for new developers.
- **NFR-M07 (M)** Dependency hygiene: lockfiles, automated update PRs, and a license check that blocks licenses incompatible with commercial distribution.

7.8 Observability

- **NFR-O01 (M)** Structured (JSON) logs with correlation IDs across devices, server and cloud. Local log rotation by size and age.
- **NFR-O02 (M)** Crash and error reporting from every component to a vendor error-monitoring service, **with personal data and secrets scrubbed** before sending.

- **NFR-O03 (M)** Health metrics are sent in heartbeats (VCP-005).
- **NFR-O04 (M)** The server PC's clock is synchronised by NTP. Devices take their time from the server, and all business timestamps use server time.

7.9 Installability and supportability

- **NFR-I01 (M)** Installing on a PC that meets the spec takes ≤ 30 minutes, excluding menu set-up.
- **NFR-I02 (M)** The installer can repair an installation. Uninstalling **keeps the data** unless the Owner explicitly chooses to delete it (password + 2FA).
- **NFR-I03 (M)** A single "Support" screen shows versions, service status, disk, backup and license state, and connected devices.

8. Security requirements

A note on "attack-proof" (Business Owner assumption A). No software can be guaranteed to be immune to attack, and a contract promising that is unenforceable. This BRD therefore defines security as **measurable compliance with recognised standards, plus an independent penetration test**, as below. This is the strongest commitment a development team can meet and verify.

8.1 Standards and baseline

Area	Standard (latest stable version at development start)
Web apps and APIs (local server, web console, QR site, relay functions, Control Plane)	OWASP ASVS Level 2 ; OWASP Top 10; OWASP API Security Top 10
Android apps	OWASP MASVS (MASVS-L1 controls + resilience controls for the kiosk tablet)
Wrist pager (IoT)	ETSI EN 303 645 baseline (no default passwords, secure updates, secure storage)

Windows host	Hardening checklist based on CIS Benchmarks (service accounts, firewall, BitLocker)
--------------	---

- **SEC-000 (M)** A **threat model** (STRIDE) is produced for each component during design and updated at each major release. It covers the LAN, the pager channel, the relay, the update pipeline and the license mechanism.

8.2 Requirements

- **SEC-001 (M) Encryption in transit everywhere.**
 - Cloud: TLS 1.2+ (1.3 preferred) with HSTS.
 - LAN: HTTPS/WSS for apps and browsers, MQTTS for pagers.
 - How LAN certificates are issued is decided in an ADR (open item OI-07). Options: (a) a per-installation private certificate authority, trusted and pinned in the vendor's own apps and installed on paired manager browsers; or (b) publicly trusted certificates for a per-restaurant hostname resolving to the LAN IP, issued via DNS validation by the Control Plane.
- **SEC-002 (M) Secrets are never in source code or in app bundles.** API keys for cloud services live only in vendor cloud secret stores and CI. Restaurant installations get **per-tenant, scoped credentials** from the Control Plane. Clients get short-lived tokens.
- **SEC-003 (M) Authorisation:** deny by default, checked on the server, including object-level checks (a tablet may act only on its own table, a waiter only on permitted tables, a pager only on its own topic).
- **SEC-004 (M) Input handling:**
 - validation at every boundary with shared Zod schemas; output encoding; parameterised queries only (Prisma), with no string-built SQL;
 - uploads checked by content type and size, images re-encoded and EXIF stripped.
- **SEC-005 (M) Cloud tenant isolation** (blueprint: per-restaurant filtering as a dedicated test):
 - Row-Level Security on every relay/Control Plane table, deny by default.
 - Local servers use per-tenant credentials, **never** a service-role key.
 - Browsers never write tables directly (QR-006).
 - **Automated cross-tenant access tests run in CI.**

Each local database holds only one restaurant, but still stores the restaurant ID on every record for consistency with the cloud.

- **SEC-006 (M) Key management:**

- The license-signing private key is in a cloud KMS/HSM and is never on a developer machine.
- Code-signing keys are on hardware tokens or cloud HSM.
- Local secrets (pepper, backup key, TLS keys) are protected by Windows DPAPI.
- Device credentials are in Android Keystore or ESP32 encrypted storage.

- **SEC-007 (M) Data at rest:**

- PostgreSQL bound to localhost, with a dedicated service account and least-privilege database roles (AUD-004).
- **BitLocker recommended** on the server PC (site-readiness checklist).
- Backups and archives encrypted with AES-256.
- Diner phone numbers encrypted at application level (S).

- **SEC-008 (M) Network:**

- Staff devices on a network **separate from guest Wi-Fi** (§ 11).
- The server firewall allows only the required ports, from the staff subnet only.
- **No inbound connections from the internet**; outbound traffic only to an allow-list of vendor/relay domains.

- **SEC-009 (M) Abuse protection:** rate limiting on login, pairing, QR submission and service requests; lockouts (AUTH-003).

- **SEC-010 (M) Mobile:**

- no secrets or personal data in logs;
- pinned certificates for server connections;
- kiosk lock (TAB-001);
- MDM policies: USB debugging off, app installation blocked, remote lock/wipe.
- Root/tamper detection on waiter devices and screenshot blocking on PIN screens are S.

- **SEC-011 (M) Desktop (Electron):**

- `contextIsolation` on, `nodeIntegration` off, sandboxed renderers;

- strict Content-Security-Policy; no remote content in privileged windows;
- IPC allow-list;
- Electron *fuses* configured (RunAsNode disabled, ASAR integrity where supported);
- Electron kept on a supported version.
- **SEC-012 (M) Pager:** PGR-005, PGR-010 and PGR-011; unique per-device credentials; broker access rules.
- **SEC-013 (M) Software supply chain:**
 - lockfiles; dependency vulnerability scanning; static analysis (SAST); secret scanning in CI;
 - an SBOM for every release;
 - protected branches; mandatory 2FA on the code host; least-privilege CI secrets;
 - builds and signing only in CI, never on developer laptops.
- **SEC-014 (M) Signed updates** (UPD-007).
- **SEC-015 (M) Security logging:**
 - Security events are audited (AUTH-013, AUD-001). No PINs, tokens or unnecessary personal data in logs.
 - Cloud security logs are kept for **at least 180 days, in India** (§ 12).
 - The Control Plane alerts on anomalies (repeated tamper events, mass failed logins, cross-tenant access attempts).
- **SEC-016 (M) Incident response plan** before the pilot:
 - severity levels, contacts and on-call;
 - regulatory reporting (CERT-In within 6 hours for reportable incidents; data-breach notifications under DPDP, § 12);
 - templates for communicating with restaurants;
 - an annual tabletop exercise.
- **SEC-017 (M) Security testing:**
 - automated authorisation tests; DAST on the QR site, relay and Control Plane;
 - an **independent penetration test** covering web, API, Android, desktop, LAN and pager, before the first paid rollout and then annually or at each major release;

- **zero open Critical/High findings at go-live**; Medium findings have a dated remediation plan.
- **SEC-018 (M) Vendor access:** MFA on all vendor systems, least privilege, quarterly access reviews, and all production access audited.
- **SEC-019 (M) Privacy by design:** collect the minimum personal data, record consent with its purpose and timestamp, and support erasure (DATA-011).

9. Data requirements

9.1 Main entities (conceptual)

Area	Entities
Restaurant	Restaurant (legal details, GSTIN, FSSAI), Settings, BusinessDay, TaxGroup, InvoiceSeries
People and access	Staff, Role, Permission, Credential (PIN hash), Session, Device (type, pairing, binding), Pager assignment
Floor	Section, Table, ShiftAssignment, TableSession (seated-at, covers, responsible waiter)
Menu	Menu version, Category, Item, Variant, ModifierGroup, Modifier, Combo, ComboSlot, Tag, Synonym, Photo, Station, Printer, StockLevel
Orders	Order (source, externalRef, idempotency key), OrderItem, OrderItemModifier, OrderEvent (status history), KOT, Approval
Service	ServiceRequest, Notification, NotificationRule, Acknowledgement, Escalation
Billing	Invoice, InvoiceLine, TaxLine, Discount, Payment, Shift, CashMovement, DayEnd
Customers	CustomerContact (optional), Consent, Feedback
Recommendations	RecommendationRule, LearnedAssociation (nightly), RecommendationEvent

Governance	AuditLog (hash chain), Backup, Archive, PurgeRecord, License, UpdateHistory
Sync	Outbox, Inbox (idempotency), SyncCursor

9.2 Data ownership and privacy roles

- **The restaurant owns its data.** Under the DPDP Act the restaurant is the *data fiduciary* for its diners' and staff personal data.
- **The vendor is a *data processor*** for data it handles for the restaurant (QR relay, cloud backups, diagnostics). It is the fiduciary only for its own customer-account data (restaurant contacts, subscription records).
- A data-processing agreement between vendor and restaurant sets this out (§ 19).

9.3 Retention (defaults ⚙)

Data	Location	Default retention	How removed
Invoices, orders, payments, tax data	Local	At least the statutory period (§ 12); kept until the Owner archives	Archive-then-purge (DATA-008)
Audit log	Local (+ chain head in cloud)	Same as invoices	Archive-then-purge
Operational logs, notifications, heartbeats	Local	90 days	Automatic (DATA-007)
Diner contact + consent	Local	Until consent is withdrawn, or 24 months without a visit	Automatic anonymisation / on request
Feedback	Local	24 months, then anonymised	Automatic
Menu photos	Local + cloud	While the item exists + 90 days	Automatic

QR orders in relay	Cloud	7 days after successful import (expired orders: 7 days)	Automatic
Cloud backups	Cloud (India)	30 daily + 12 monthly	Automatic; deleted within 30 days after contract termination, after notice and an export offer
Diagnostics bundles	Cloud	30 days	Automatic
Control Plane security and access logs	Cloud (India)	≥ 180 days	Automatic

9.4 Data rules for developers

- **Money is stored as integer paise**, never as floating-point numbers. Rounding rules are explicit and tested.
- Timestamps are stored in UTC and shown in IST. Every business record also carries its **business date**, because night service after midnight belongs to the previous day until the cut-off time.
- Soft-delete/archive for master data; no hard deletes for transactional data (AUD-004).
- Referential integrity is enforced in the database. Every migration is reviewed and reversible by restore (UPD-005).
- All cloud data is stored and processed **in India**.

9.5 Data migration

Greenfield: there is no legacy data migration. Initial data comes from the onboarding wizard and menu import (ONB-004 to ONB-007).

10. Technical architecture and recommended stack

10.1 Architecture principles

1. **Local-first**: the local server is the single source of truth for the restaurant. The cloud is an extension, never a dependency for service.

2. **Modular, ports and adapters** (INT): isolated modules, contract-first, integration-ready.
3. **Event-driven real time**: state changes emit domain events. Events reach apps via Socket.io and pagers via MQTT. Cross-boundary delivery uses an outbox/inbox with idempotency keys.
4. **Secure by default** (§ 8).
5. **Everything versioned**: APIs, events, menus, licenses, firmware, database schema.
6. **One UI codebase per platform family**: web (POS, dashboard, KDS) and React Native (waiter app, tablet), sharing design tokens and domain logic.

10.2 Recommended stack

The blueprint's stack is retained. Changes are marked **Changed** or **Added**, with reasons.

Concern	Recommendation	Status vs. blueprint
Language	TypeScript, <code>strict</code> , everywhere except pager firmware	Same
Monorepo	Turborepo + pnpm workspaces	Same
Shared contracts	Zod package, generating OpenAPI + AsyncAPI docs and typed API clients	Extended
Local server	NestJS on Node.js LTS; Prisma; Socket.io; Aedes MQTT broker (TLS + auth/ACL hooks); bcrypt/Argon2id	Changed : runs as its own Windows service (service wrapper such as WinSW) with a watchdog, separate from the UI. Otherwise closing the POS window would stop every tablet and pager.
Local database	PostgreSQL installed as a Windows service, data on the chosen drive, localhost only	Same (embedded-postgres or bundled binaries, whichever runs reliably as a service)
POS, Manager Dashboard, KDS	One React web console served by the local server, with role-based modes; Electron shell on the restaurant PC (electron-builder, electron-updater)	Changed : POS and dashboard unified, and the KDS is a web mode instead of a separate Electron app. KDS then runs on cheap Android tablets/TVs, and the same screens work on the manager's phone.

Waiter app, table tablet	React Native + Expo (development builds), EAS Build + EAS Update (with code signing); kiosk via Android device-owner lock-task + MDM	Extended
Voice search	Android on-device SpeechRecognizer through an actively maintained module (e.g., <code>expo-speech-recognition</code>); check maintenance status before choosing <code>@react-native-voice/voice</code>	Changed
Menu search	fuse.js plus synonym and tag index	Extended
QR site	Next.js, hosted on a platform with India presence and CDN	Same
Cloud relay	Supabase in the Mumbai region: Postgres with RLS, Edge Functions for all writes, Realtime, Storage (photos)	Extended
Vendor Control Plane	NestJS + React (reusing monorepo packages), managed Postgres in India	Added
Pager firmware	ESP32-S3; ESP-IDF (or Arduino-ESP32 on ESP-IDF) with esp-mqtt (TLS, QoS 1); secure boot v2; flash encryption; signed OTA with rollback	Changed: PubSubClient cannot publish at QoS 1, which reliable acknowledgements need.
Charts	Recharts	Same
Report exports	Server-side PDF rendering and XLSX generation (e.g., ExcelJS)	Added

Printing	node-thermal-printer (network) + USB via Windows spooler/raw printing	Extended
Testing	Vitest (logic), Playwright (web/Electron), Maestro (Android); plus contract tests, load tests (k6 or similar), firmware hardware-in-the-loop tests	Extended
CI/CD	GitHub Actions + code scanning (SAST), dependency scanning, secret scanning, SBOM, signing steps, release promotion to channels	Extended
Error monitoring	Sentry or equivalent, with PII scrubbing	Added
Device management	Commercial Android MDM (vendor to be selected, OI-05)	Added

10.3 Proposed repository structure

The folder list in the blueprint was missing; this is a proposal the lead may refine.

```

/apps
  /server          NestJS local server (Windows service)
  /console         React web console: POS, manager dashboard, KDS
modes
  /desktop         Electron shell + installer config
  /waiter-app      React Native (Expo)
  /table-tablet    React Native (Expo)
  /qr-menu         Next.js public site
  /control-plane   Vendor Control Plane (API + web)
/packages
  /contracts       Zod schemas, API + event definitions,
OpenAPI/AsyncAPI generation
  /domain          Framework-free business logic: pricing, tax, state
machines, recommendations
  /api-client      Typed client SDK used by every app
  /ui-web          Web component library
  /ui-native       React Native component library
  /design-tokens    Shared colours, typography, spacing
  /licensing       License verification (shared by server and apps)
  /i18n            String catalogues (English only in v1)
  /config          Shared ESLint, TS, Vitest configs
/firmware
  /pager           ESP-IDF project (built by its own CI job)
/infra
  /supabase        Migrations, RLS policies, Edge Functions
  /installer       Windows installer scripts, service wrappers,
watchdog
  /ci              Pipeline definitions, signing scripts
/docs
  /adr  /runbooks  /api

```

10.4 Communication

Path	Mechanism
Apps ↔ local server (commands, queries)	HTTPS REST

Local server → apps (live updates, alerts)	Socket.io over WSS, with rooms per table, section, station and role
Local server ↔ pagers	MQTT over TLS (Aedes broker), per-device topics, QoS 1
Local server ↔ QR relay	Outbound HTTPS + Realtime subscription with polling fallback; outbox/inbox with idempotency keys (blueprint)
Local server ↔ Vendor Control Plane	Outbound HTTPS: heartbeat, license renewal, update manifests, diagnostics upload
Inside the server	In-process domain event bus with a persistent outbox for reliable cross-module delivery
Device discovery	Configured server address (DHCP reservation); mDNS discovery is S

10.5 Environments

Development, Staging and Production for cloud components. For local components: a **physical "lab restaurant" rig** (PC, 2 tablets, 2 phones, 1 KDS, 2 printers, 4 pagers, business router) used for integration, firmware and failure testing. Feature flags are used for incremental rollout.

11. Hardware and site requirements

The vendor publishes and maintains a **supported-hardware list**; only listed models are supported. Final models are open item OI-04.

Item	Minimum	Recommended / notes
Restaurant PC (server + POS)	Windows 11 64-bit; 4-core CPU (e.g., Intel Core i3 10th gen / Ryzen 3 or newer); 8 GB RAM; 256 GB SSD; wired Ethernet	16 GB RAM; second drive (or external USB/NAS) for backups; touchscreen monitor; BitLocker enabled
UPS	≥ 15 min runtime for PC, router and printers	Signalling UPS for graceful shutdown (S)

Receipt / KOT printers	80 mm ESC/POS thermal, Ethernet	58 mm supported; USB supported for the POS printer; one printer per printing station
KDS screens	Android tablet $\geq 10"$ or TV/monitor with Android box, recent Chromium browser	Wall/arm mount; splash protection near cooking areas
Table tablets	Android 11+, 8-10", ≥ 3 GB RAM, supports Android Enterprise device-owner mode	Anti-theft stand or lock; charging solution at table or charging rack
Waiter phones	Android 11+, ≥ 3 GB RAM, restaurant-owned, MDM-enrolled	Rugged case; spare device
Wrist pagers	Per PGR-001 to PGR-003	Charging dock for all pagers + 20 % spares
Network	Business-grade router/access points supporting separate SSIDs/VLANs (staff vs guest) , WPA2/WPA3, DHCP reservations	Good 2.4 GHz coverage everywhere pagers go (kitchen, pass, floor, terrace), because ESP32 pagers use 2.4 GHz only. Mesh APs for large floors. Site survey before installation.
Internet	Broadband ≥ 10 Mbps	4G failover router (S)

12. Compliance and regulatory requirements (India)

The platform provides the *capabilities* for compliance. Legal interpretation for each restaurant stays with its CA/lawyer, and for the vendor with its counsel. Items marked "confirm" need professional confirmation before launch.

Area	What applies	Platform impact	Owner
GST invoicing (CGST Act and Rules)	Tax-invoice particulars; invoice numbers unique and consecutive per financial year, max 16 characters	BILL-001 to BILL-004, BILL-007, RPT-006	Dev team; restaurant CA confirms templates

GST tax rates	Rates depend on restaurant type and change over time	Configurable tax groups only; nothing hard-coded (BILL-004)	Restaurant CA
GST record retention (Section 36, CGST Act)	Accounts/records kept 72 months from the due date of the annual return for that year	DATA-008 purge warnings; archive retained by restaurant	Restaurant; vendor provides tooling
GST e-invoicing (B2B)	Depends on restaurant turnover	Out of v1; customer GSTIN captured (BILL-011); confirm	Restaurant CA / Business Owner
FSSAI	License/registration number printed on bills	BILL-002	Dev team
Service charge	CCPA guidelines (2022), upheld by the Delhi High Court (March 2025): service charge cannot be mandatory or added by default	BILL-006 (off by default, labelled voluntary, removable)	Dev team; restaurant
Personal data (DPDP Act 2023 and DPDP Rules 2025)	Rules notified 14 Nov 2025 with an 18-month phased compliance timeline. Notice and consent, purpose limitation, erasure, breach notification, processor obligations.	TAB-014, BILL-011, DATA-011, SEC-016, SEC-019, § 9.2	Business Owner (legal review); dev team
Cyber-incident reporting (CERT-In Directions, 2022)	Report specified incidents within 6 hours; keep logs 180 days in India; synchronise clocks to NTP	SEC-015, SEC-016, NFR-004	Business Owner (point of contact); dev team
Commercial SMS	TRAI DLT registration of entity, sender ID and templates	Only if SMS e-bills are used (BILL-012)	Business Owner
WhatsApp messages	WhatsApp Business Platform via a provider; approved templates	Only if WhatsApp e-bills are used	Business Owner

Wireless device (pager)	WPC Equipment Type Approval for Wi-Fi devices; possibly BIS Compulsory Registration (smart watches are listed under IS 13252 Part 1); battery safety. Confirm with a compliance consultant.	PGR-001 (pre-certified module)	Business Owner
Open-source licenses	Commercial distribution must respect each dependency's license	NFR-M07	Dev team; Business Owner signs off

13. Assumptions

ID	Assumption
A-01	Each owner runs a single outlet . There is one installation (one local server) per outlet.
A-02	The restaurant provides a Windows PC meeting § 11, kept powered during business hours with a UPS.
A-03	The restaurant has, or will install, a staff Wi-Fi network separated from guest Wi-Fi, with 2.4 GHz coverage across floor, pass and kitchen. The vendor provides a site survey checklist.
A-04	Broadband internet is usually available but not guaranteed. Core operations don't depend on it.
A-05	Easy onboarding with a custom menu: each restaurant's menu is its own (no shared global catalogue). The restaurant supplies it as Excel, PDF or photos, along with table, staff and tax details. The vendor's onboarding team assists, remotely or on-site.
A-06	The restaurant's CA provides tax rates and invoice requirements. The platform doesn't give tax advice.
A-07	English is acceptable to staff and diners for v1.

A-08	Staff have basic smartphone skills; about 30 minutes of training per role is enough.
A-09	Restaurants buy the complete ecosystem . Partial module sales are not offered in v1.
A-10	All staff and table devices are restaurant-owned models from the supported-hardware list, enrolled in MDM. There is no BYOD.
A-11	Payments are collected outside the platform; staff record the payment mode manually.
A-12	Service types are dine-in and takeaway only; QR ordering is dine-in only.
A-13	The restaurant owns its data; the vendor processes it only under contract.
A-14	The owner views reports on the restaurant's system or receives exported files from staff.
A-15	The development team is experienced in all technologies listed, including embedded firmware and Windows packaging.
A-16	At least one pilot restaurant will be available for field testing before commercial launch.
A-17	The Business Owner provides vendor accounts, certificates and legal documents on time (§ 19).
A-18	Restaurants are within the capacity targets in § 7.2.

14. Constraints

- **Local-first:** the operational database lives on the restaurant's PC, and nothing on the internet connects into the restaurant.
- **Platforms:** Windows (restaurant PC), Android (phones, tablets, optionally KDS), ESP32 (pager). No iOS apps.

- **Distribution:** Android apps are distributed as signed APKs through MDM / in-app updater, not the public Play Store.
- **Radio:** pagers use 2.4 GHz Wi-Fi, which constrains site networking.
- **Technology:** the blueprint's stack, with changes only through an ADR.
- **Regulation:** Indian tax, consumer-protection, data-protection and telecom rules (§ 12).
- **Commercial:** budget and dates are set by the Business Owner and are outside this document.

15. Dependencies

Dependency	Needed for	Risk if late
Supabase (relay), web hosting, managed Postgres (India)	QR menu, Control Plane	QR and licensing blocked
Expo / EAS	Android builds and OTA updates	Mobile releases blocked
Code-signing certificate	Trusted Windows installer and updates	Windows security warnings on every install; auto-update trust broken
MDM provider	Kiosk tablets, APK distribution	Tablets not lockable; manual updates
Pager hardware (prototype units)	Battery gate (PGR-002), firmware development	Pager track slips
Supported tablets, phones, printers, router for the lab rig	Integration and failure testing	Late discovery of hardware issues
AI provider API	AI menu import (S)	Fall back to Excel import
SMS/WhatsApp provider (+ DLT registration)	Digital bills (S)	Feature deferred
Legal documents (subscription agreement, DPA, privacy notices)	Pilot and sales	Cannot legally onboard

Penetration-testing firm	Go-live	Launch blocked by SEC-017
Pilot restaurant	Field validation, KPI baselines	Unvalidated launch

16. Risks and mitigations

ID	Risk	Likelihood	Impact	Mitigation	Owner
R-01	Pager battery can't last a shift with Wi-Fi always on	High	High	Prototype battery gate in Phase 0 (PGR-002); Wi-Fi power-save; charging dock + spares; evaluate an alternative radio or base-station design if the gate fails	Dev lead
R-02	Wi-Fi dead zones in kitchens (metal, heat, microwaves)	Medium	High	Site survey; recommended APs; pager offline detection and manager fallback (NTF-007)	Onboarding team
R-03	Restaurant PC fails during service	Medium	High	UPS; watchdog; backups; restore runbook (DATA-004); spare-PC plan; future hot-standby	Dev lead / Business Owner
R-04	Mandatory waiter approval slows service at peak	Medium	Medium	Approve in ≤ 2 taps, including PIN on the tablet at the table; escalation to manager; approval-time KPI; revisit per-restaurant auto-approval after the pilot	Business Owner

R-05	License cracked or patched	Medium	Medium	Signed binaries, integrity checks, machine binding, clock checks; real leverage is cloud services (QR, updates, backups, support); contract terms	Dev lead / Business Owner
R-06	Staff resist new workflow	Medium	High	Usability testing, training videos, pilot "champion", simple UI	Business Owner
R-07	An update breaks a restaurant	Low	High	Maintenance window, staged rollout, auto-rollback, per-restaurant pinning, N-1 compatibility	Dev lead
R-08	Cross-tenant data leak or breach in cloud	Low	High	RLS, scoped credentials, CI isolation tests, pentest, least privilege	Dev lead
R-09	Voice search misrecognises Indian dish names	High	Low	Synonyms, fuzzy + tag search; voice is S, not M; typing always available	Dev lead
R-10	Data lost through purge or failed backup	Low	High	Archive-verify-then-purge; Owner 2FA; backup monitoring and alerts; restore drills	Dev lead
R-11	Tax or data-protection rules change	Medium	Medium	Configurable tax; legal reviews; fast update pipeline	Business Owner
R-12	Cloud provider outage	Low	Medium	Local-first design; QR degrades to read-only	Dev lead

R-13	Tablet theft or damage	Medium	Low	MDM remote lock/wipe; security stands; no customer data left after a session	Restaurant / Onboarding
R-14	Pager certification delays commercial launch	Medium	Medium	Engage a compliance consultant early; use a pre-certified radio module	Business Owner
R-15	Scope creep	Medium	Medium	Change control against this BRD; decisions logged (Appendix A)	Business Owner
R-16	Dispute over license restriction	Low	High	Staged states, never mid-service, data always exportable; terms in contract	Business Owner

17. Release plan

Durations are estimated by the development team after sign-off. Each phase ends with a demonstrable, tested increment. The order follows the blueprint (core → waiter/pager → tablet → dashboard → QR → advanced menu → voice → packaging/licensing), with the update pipeline, security baseline and hardware risk checks brought forward to Phase 0.

Phase	Scope	Exit criteria
0: Foundations and risk spikes	Monorepo, contracts package, CI/CD with security scanning and signing, design system, Windows service + installer skeleton, PostgreSQL setup, auth + device pairing, LAN TLS ADR, minimal Control Plane (heartbeat + update channel). Spikes: pager battery prototype, Wi-Fi coverage test, kiosk mode on the chosen tablet, printer compatibility.	Installer installs and updates itself on a clean PC. Pager battery gate (PGR-002) passed, or an alternative approved by the Business Owner.

1: Core POS and kitchen	Menu data model (including variants, modifiers, combos), tables and sessions, POS order entry, KDS + station routing + printing, billing and GST, manual payments, shift/day-end, audit trail, basic reports, takeaway	A simulated full service day on POS + KDS with correct invoices and audit trail
2: Waiter app, notifications, pagers	Waiter app, notification engine + escalation, pager firmware + pairing + OTA, move table, manager nudge, "Notify manager"	Alert latency and escalation targets met on the lab rig
3: Table tablet and recommendations v1	Kiosk app, session lifecycle, Water/Waiter/Bill/Cancel, ordering with approval (app + at-table PIN), live status, rule-based + best-seller recommendations, text search, feedback	Tablet order flow end to end with no data left over between sessions
4: Manager dashboard and reports	Live views, staff/device/menu management, configuration screens, full report suite and exports, suspicious-activity report	All RPT (M) reports verified against test data
5: QR menu and cloud relay	Next.js site, relay with Edge Functions + RLS, sync agent, outbox, photo mirroring, statuses and timeouts, read-only mode	QR scenario tests pass, including internet cut and duplicate replay
6: Advanced menu and intelligence	Modifiers/combos UI on all surfaces, learned recommendations, voice search, AI menu import	Same combo + modifier order succeeds on every surface (blueprint week 30 test)
7: Commercial readiness	Full license states, Control Plane (tenants, subscriptions, fleet, releases), backup/restore/archive/purge, onboarding wizard completion, digital bills	License and data scenarios S8 and S10 pass
8: Hardening and pilot	Load tests at capacity, failure drills, penetration test + fixes, UAT, pilot in 1-2 restaurants for ≥ 4 weeks ⚙️, go-live checklist	\$ 18 acceptance met; pilot KPIs measured; zero open Critical/High security findings

18. Testing and acceptance

18.1 Test levels

Unit (domain logic), integration (server modules and database), contract (INT-002), end-to-end (Playwright for web/desktop, Maestro for Android), firmware hardware-in-the-loop, load/performance (§ 7.2), security (§ 8), usability (§ 7.4), and the pilot.

18.2 Critical scenarios (all must pass before the pilot)

ID	Scenario
S1	Dinner rush: 40 tables, 3 stations, 8 waiters, 25 tablets, 10 pagers, 600 orders in 3 hours (simulated + real devices). All § 7.1 targets met; zero lost or duplicate orders/KOTs.
S2	Internet outage for 30 min mid-service: in-restaurant flows unaffected; QR read-only; queues drain after recovery with no duplicates.
S3	Server restart mid-service: clients reconnect ≤ 30 s; no lost orders or alerts.
S4	Printer failure: alert raised, KOTs queued/redirected, nothing lost.
S5	Pager out of range / battery dead: alerts reach the waiter app; if both are unavailable, the manager is alerted immediately.
S6	Tablet session: open → order → approval (app and at-table PIN) → status → Water/Cancel → bill → reset; nothing carried over.
S7	Move table with tickets in progress: tickets updated in place, not duplicated.
S8	License: overdue → grace → restricted (only at next business day) → reactivation online and offline; data export works in restricted mode.
S9	Update failure: forced failed health check → automatic rollback; restaurant keeps working.
S10	Backup and restore to a new PC; archive-then-purge with verification; archive viewer.
S11	Cross-tenant isolation in the relay and Control Plane (automated).
S12	Fraud patterns: void after print, repeated reprints, over-limit discount, KOT never billed. All appear in reports with the correct people attributed.

S13	QR order with restaurant offline: customer sees "please call a waiter" after T seconds; a late-arriving order is rejected as expired.
S14	Escalation: unacknowledged alert reaches the manager after N seconds; acknowledgement stops repeats.

18.3 Acceptance

- Every **M** requirement is linked to at least one passing test, in a traceability matrix maintained by the team.
- Defect severity: **Sev-1** service stopped or data lost/incorrect money; **Sev-2** major function impaired, no workaround; **Sev-3** workaround exists; **Sev-4** cosmetic.
- **Pilot entry:** all S1-S14 pass; no open Sev-1/Sev-2; penetration test with no open Critical/High findings.
- **Commercial launch:** pilot $\geq$ 4 weeks ☼ with no Sev-1 in the last 2 weeks; KPIs (§ 2) measured and reviewed; Business Owner sign-off.

19. Business Owner responsibilities

These are the items the Business Owner (vendor) must arrange; the development team can't provide them.

Important correction to a common assumption: API keys and secrets must **not** be written into the application code. Anything inside an APK or .exe can be extracted. Keys are held in the vendor's cloud secret stores and CI; restaurant installations receive their own scoped credentials from the Control Plane (SEC-002). The Business Owner's job is to **own the accounts** and give the development lead controlled access to them.

19.1 Accounts, subscriptions and keys

#	Item	Why	Needed by
1	Company-owned GitHub organisation (paid plan), 2FA enforced	Code, CI/CD	Phase 0

2	Domain name(s) + DNS provider	QR site, Control Plane, update server, LAN certificate approach (SEC-001)	Phase 0
3	Windows code-signing certificate in the company name (OV or EV)	Trusted installer and updates. Company verification can take weeks, so start early . The key is kept on a hardware token or cloud HSM.	Phase 0
4	Cloud KMS / secrets manager	License-signing key, backup-key escrow, CI secrets	Phase 0
5	Cloud hosting in India for the Control Plane, update distribution and QR site (e.g., AWS/GCP/Azure Mumbai, or Vercel + managed Postgres)	Vendor cloud components	Phase 0 (minimal), Phase 5/7 (full)
6	Error-monitoring service (e.g., Sentry)	NFR-002	Phase 1
7	Expo account (paid plan sized for build and OTA volume)	Android builds, OTA updates	Phase 2
8	Android app-signing keystore held under company ownership, backed up in a secure vault	Losing it means installed apps can never be updated	Phase 2
9	MDM subscription for tablets and phones	Kiosk mode, APK distribution, remote lock/wipe	Phase 3
10	Supabase organisation : separate dev/staging/prod projects, paid plan for production, Mumbai region	QR relay, photo storage	Phase 5 (dev earlier)
11	Backup storage (S3-compatible, India region)	Encrypted cloud backups	Phase 7
12	AI provider API account	AI menu import (S); key stored only in the Control Plane	Phase 6

13	SMS provider with TRAI DLT registration and/or WhatsApp Business Platform via a provider	Digital bills (S)	Phase 7
14	Transactional e-mail service	Control Plane notifications	Phase 7
15	Payment gateway merchant account (only if automating subscription collection, VCP-004)	Collecting subscriptions	Phase 7 (S)

No Apple developer account is needed (no iOS apps).

19.2 Legal, compliance and commercial

- Company incorporation and the company's own GST registration; how you will invoice subscriptions.
- **Subscription agreement / EULA** covering:
 - plans and payment terms;
 - **license states and timelines** (LIC-005), matching the software exactly;
 - data ownership and export rights; support hours and SLA;
 - limitation of liability; termination and data return.
- **Data Processing Agreement** (restaurant as fiduciary, vendor as processor); vendor privacy policy; **diner privacy notice** for the tablet and QR menu; consent wording.
- **DPDP compliance review** with counsel. The Rules' 18-month phase-in (from 14 Nov 2025) ends around your launch window.
- CERT-In point of contact and ownership of the incident-response plan (SEC-016).
- Product name and **trademark** (OI-01).
- Sign-off on open-source license compliance.
- **Pager certification** (WPC ETA; BIS CRS applicability; battery safety) via a compliance consultant, **started early**.
- Engage a **penetration-testing firm** (a CERT-In empanelled auditor is advisable).
- Cyber and product-liability insurance (recommended).

19.3 Product and operations

- Sign off this BRD. Name **one product decision-maker** for day-to-day questions from the team.

- Decide **pricing and plans**.
- Approve the **supported-hardware list**, and procure the **lab rig** and **pager prototypes early** (they gate Phase 0).
- Recruit **pilot restaurant(s)**, sign pilot agreements, and **measure baseline KPIs before go-live** (§ 2).
- Set up the **onboarding and support team**: support hours, phone/WhatsApp support line, escalation path, SLAs, and who answers at 9 pm on a Saturday.
- Approve the Excel menu template, training videos and user manuals.
- Name the **release approval authority** for production rollouts.
- Resolve the open items in § 20 by the phase shown.

20. Open items

ID	Item	Default until decided	Owner	Needed by
OI-01	Product name and branding (logo, colours for the design system)	"Restaurant Operations Platform"	Business Owner	Phase 0
OI-02	Escalation timeout N and repeat interval R	N = 60 s, R = 60 s; tune in pilot	Business Owner	Phase 2
OI-03	Bar/liquor (VAT) billing needed for the pilot?	Excluded	Business Owner	Before Phase 1
OI-04	Exact hardware models (PC, tablet, phone, printer, router, pager)	Per § 11	Business Owner + dev lead	Phase 0
OI-05	MDM vendor	To be selected	Dev lead → Business Owner	Phase 3
OI-06	SMS vs WhatsApp (or both) for digital bills	WhatsApp preferred	Business Owner	Phase 7

OI-07	LAN TLS approach (SEC-001)	ADR by dev lead	Dev lead	Phase 0
OI-08	Grace period, overdue thresholds, subscription terms	7-day grace	Business Owner + counsel	Phase 7
OI-09	First third-party POS for future integration	None (future)	Business Owner	After v1
OI-10	Cloud backup key-escrow policy (DATA-003)	Escrow, released on Owner's authenticated request	Business Owner + counsel	Phase 7
OI-11	Can kitchen staff mark items out of stock?	Yes	Business Owner	Phase 1
OI-12	Service charge default for new restaurants	Off	Business Owner	Phase 1
OI-13	The supplied blueprint seems to be missing sections (Phase 1-2 headings, weeks 1-17, 20-22, 32-33, the monorepo folder list, the Week 34 title). Do they hold requirements not captured here?	Assume none	Business Owner	Before sign-off

21. Glossary

Term	Meaning
ADR	Architecture Decision Record: a short document recording a significant technical decision and why
BYOD	Bring Your Own Device
CA	Chartered Accountant
Captain	Senior waiter who takes orders (used interchangeably with waiter here)

Covers / pax	Number of guests at a table
DPDP	Digital Personal Data Protection Act 2023 (India) and its Rules
ESC/POS	Command language used by most thermal receipt printers
ESP32	Low-cost Wi-Fi microcontroller used in the wrist pager
Idempotency key	Unique ID on a request, so a retried request is processed only once
KDS	Kitchen Display System
KOT	Kitchen Order Ticket: the list of items for a kitchen station
MDM	Mobile Device Management: remote management and locking of Android devices
MQTT	Lightweight messaging protocol, used for the pagers
OTA	Over-the-air (remote) update
Outbox / inbox	Pattern where messages to/from the cloud are stored locally first and retried until confirmed
RLS	Row-Level Security: database rules that restrict which rows each tenant can see
RPO / RTO	Recovery Point Objective (how much data may be lost) / Recovery Time Objective (how long recovery may take)
SAC	Services Accounting Code, used on GST invoices for services
Station	A kitchen section (tandoor, Chinese, bar, desserts) with its own screen and/or printer
Tenant	One restaurant customer in the vendor's cloud
Z-report	End-of-day sales and cash summary

Appendix A: Decision log

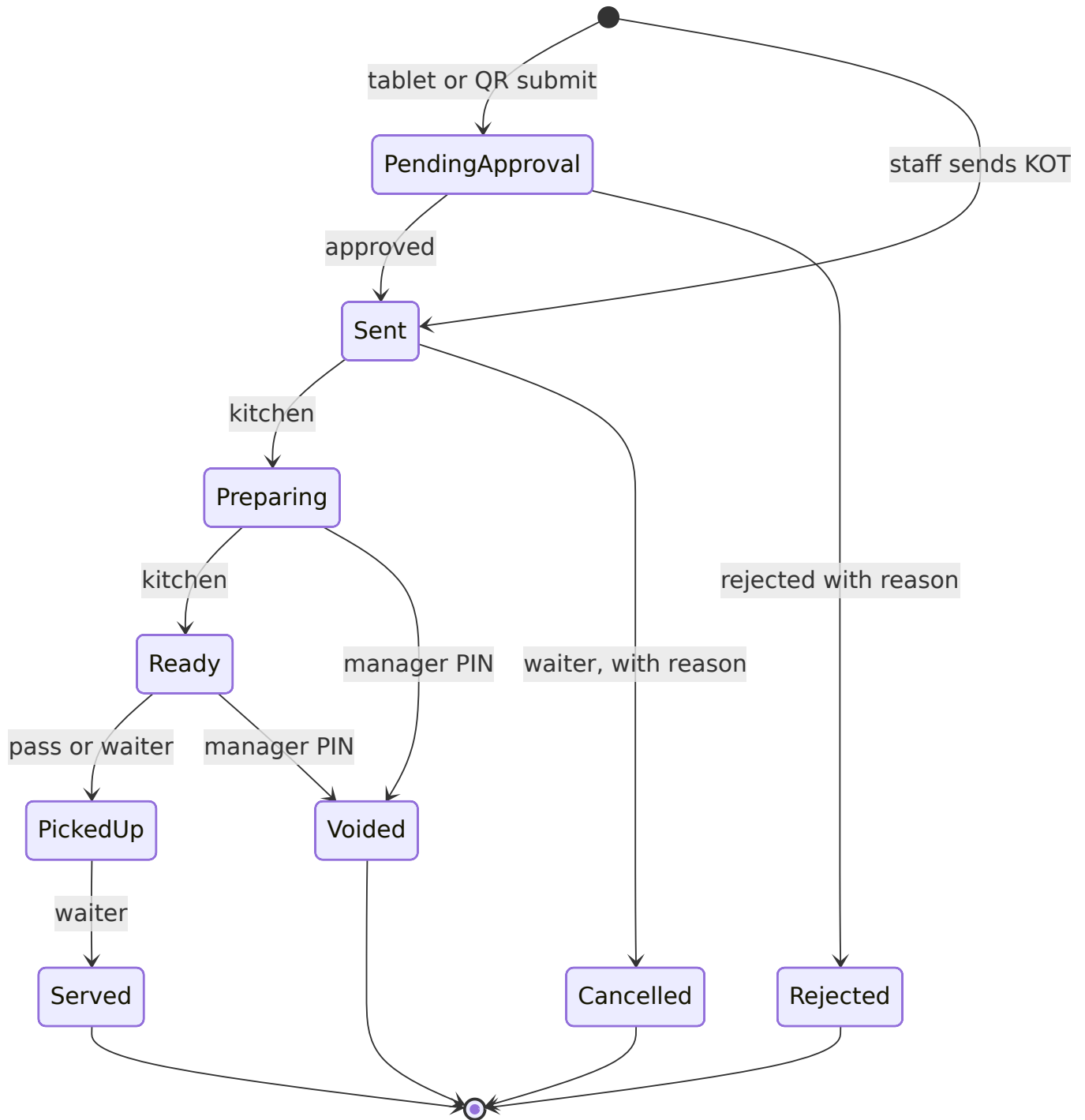
Decisions from the requirements workshop of 25 Sep 2026. "Default" means the Business Owner accepted the recommended option.

Q	Topic	Decision
1	Market and formats	India only for v1; dine-in and takeaway; bar billing only if the pilot needs it (OI-03); tax/currency configurable. (Default)
2	Chains and owner access	One outlet per owner. The owner accesses reports on the restaurant's system; staff can export and send them. No cloud owner dashboard.
3	Aggregators and inventory	Out of v1; data model keeps hooks. (Default)
4	Payments	No payment integration. Payment mode recorded manually. (UPI QR on bill moved to future scope.)
5	Customer data	Optional, consent-based phone capture; digital bill; feedback. Loyalty later. (Default)
6	Languages	English only for now (UI strings externalised).
7	Tablet orders	Waiter approval required; the waiter is notified.
8	Tablet session	Opened by waiter/POS; wiped on close; live item status shown. (Default)
9	Cancel button	Cancels the table's active service request(s) (Water / Waiter / Bill), like a flight-cabin call button: pressed by the waiter on arrival, or by the diner if no longer needed.
10	QR ordering	Dine-in only. Waiter approval makes separate anti-remote-order checks unnecessary. No payment before ordering.
11	Kitchen stations	Item → station routing; per-station screen and/or printer. (Default)
12	Escalation	An unacknowledged alert goes directly to the manager after N seconds (no backup waiter in between).
13	Recommendations	Three layers: manual rules → learned "ordered together" → best sellers; table-level only. (Default)

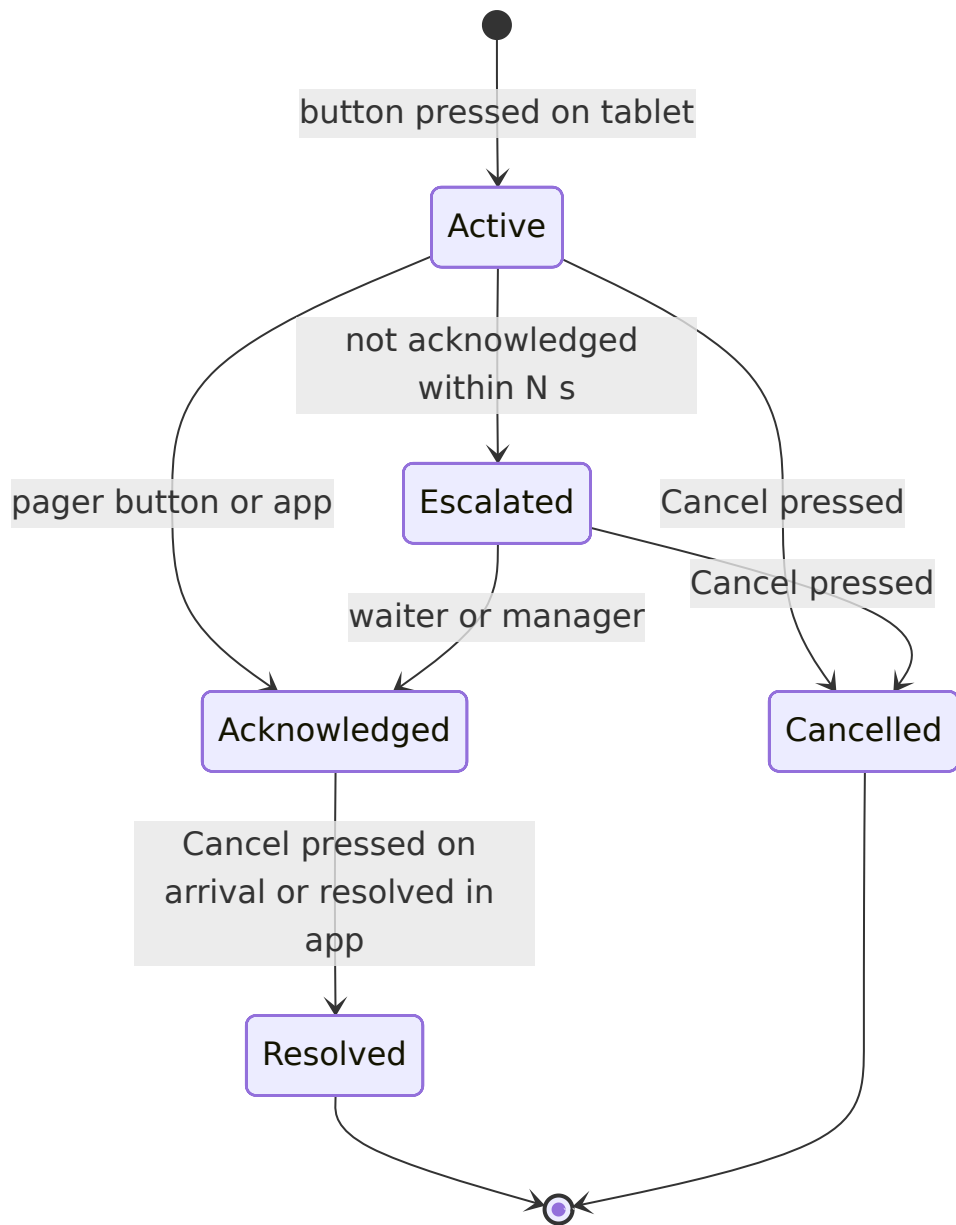
14	Module-only installs	Superseded by Q15: not in v1.
15	Other POS / separate sale	Complete ecosystem only in v1. Code built as isolated modules with documented contracts so integration with other POS is possible later. First POS undecided.
16	Data clearing	Archive, then purge; operational data auto-purged; disk alerts. (Default)
17	Data drive and backups	Installer lets the restaurant choose the drive (default non-system); nightly local + encrypted cloud backup; one-click restore. (Default)
18	Kill switch	Staged: reminders → 7-day grace → restricted mode; never mid-service; data never withheld. (Default)
19	Offline license	Signed license valid 30 days, renewed silently when online. (Default)
20	Remote fixes and support	Channels, per-restaurant pinning, staged rollout, rollback, backup before update, maintenance window; consent-based diagnostics; no hidden remote access. (Default)
21	Pager	Off-the-shelf ESP32 watch for pilot; vibrate, LED, display, acknowledge button. (Default)
22	KDS	Web app on inexpensive hardware instead of Electron. (Default)
23	Hardware supply	Supported-hardware list; optional bundles; tablets in kiosk mode under MDM. (Default)
24	Team	Competent team; full scope; no reduced MVP needed.
-	Additional recommendations accepted	Server as a Windows service; unified POS/dashboard codebase; voice-search tags/synonyms and library check; cloud tenant isolation focus; anti-fraud audit; GST billing; shift/day-end; onboarding wizard + AI menu import; device pairing; separate staff network; design system; error monitoring; Vendor Control Plane; security standards + penetration test.

Appendix B: State machines

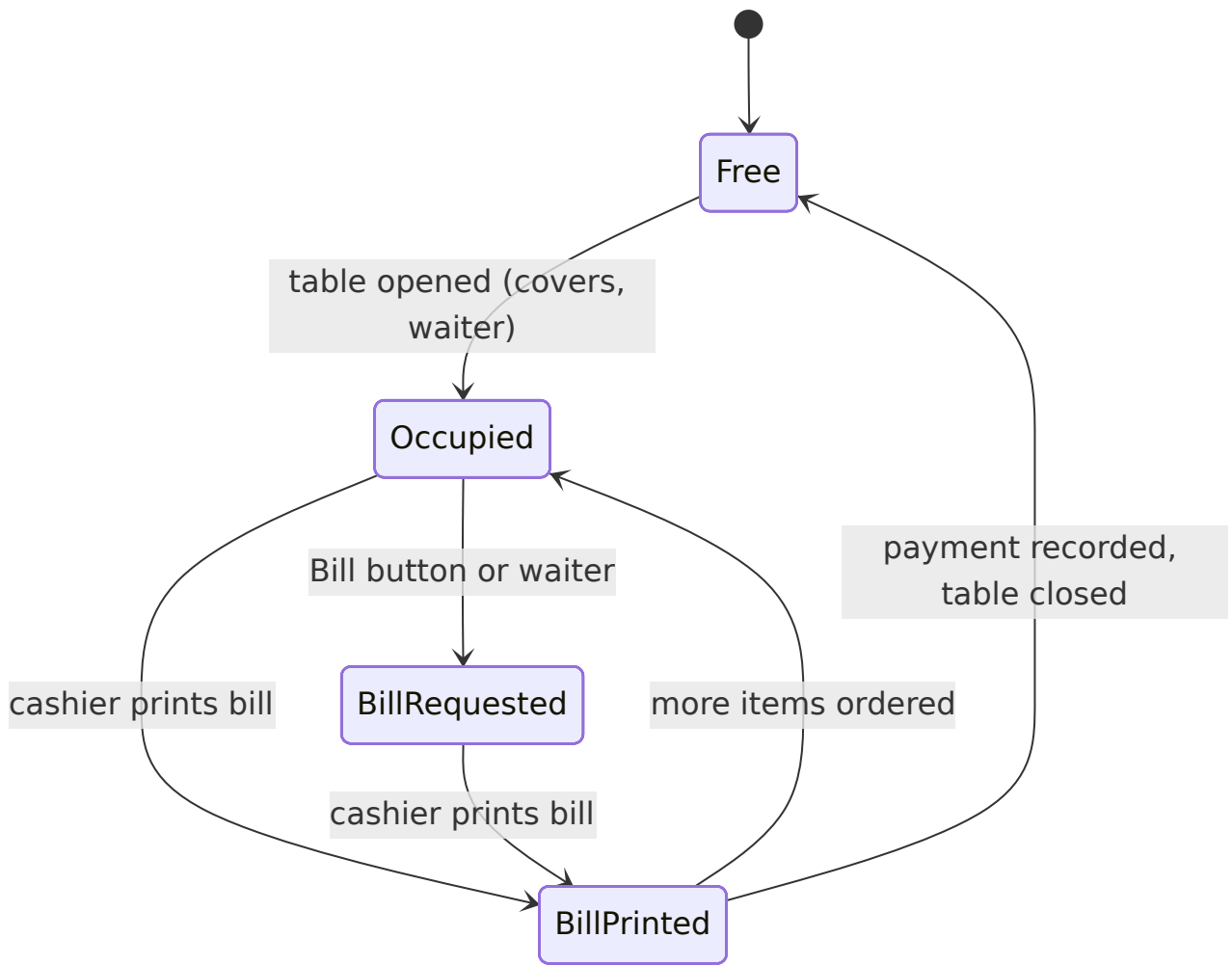
Order item



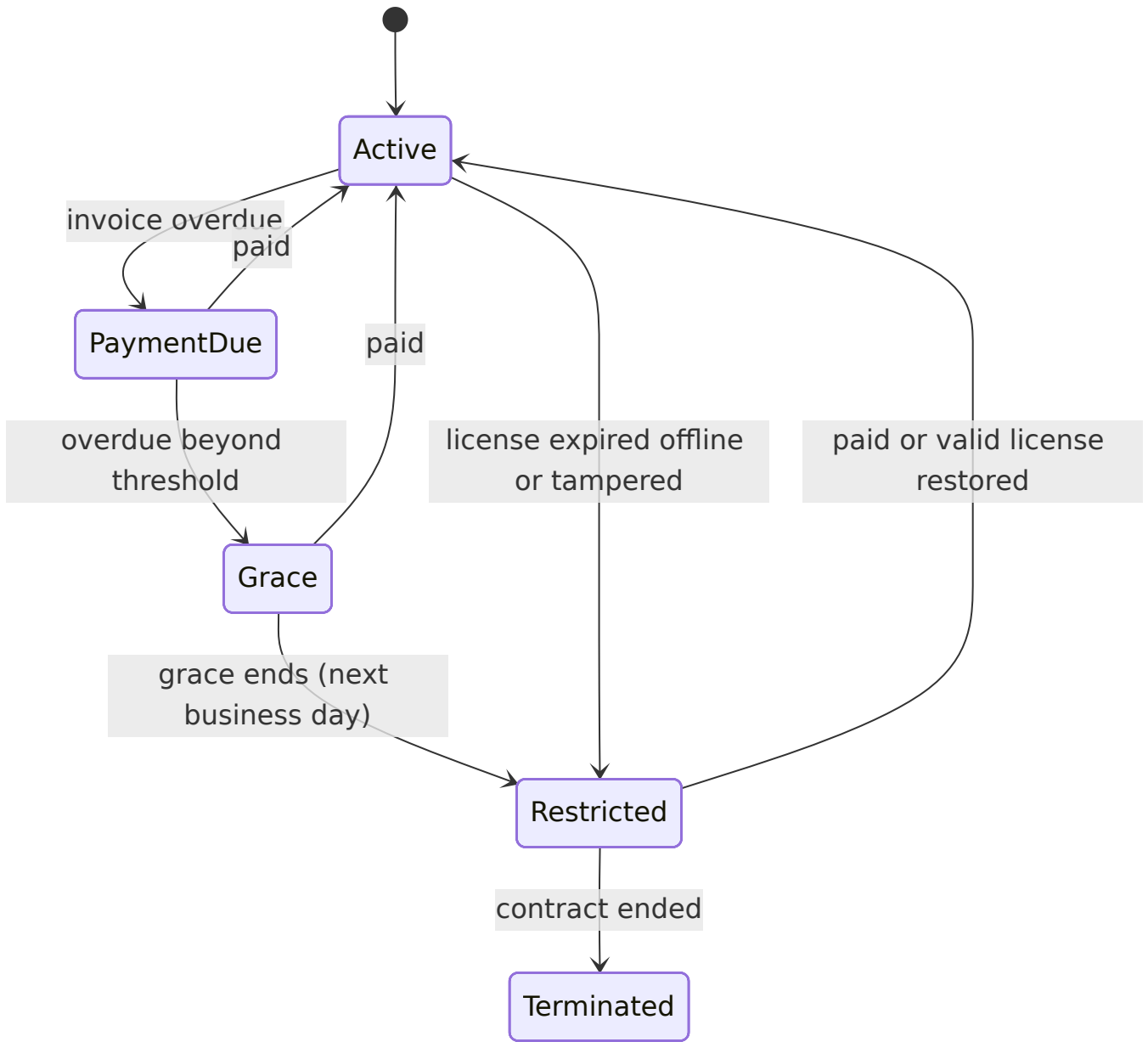
Service request (Water / Waiter / Bill)



Table



License



Appendix C: Default notification matrix

N (escalation) and R (repeat) are configurable (NTF-002); factory defaults N = 60 s, R = 60 s.

Event	Primary recipient(s)	Channels	Pager text	If not acknowledged in N s	Repeat
Customer order pending approval	Responsible waiter	Pager, waiter app, POS badge	T5 NEW ORDER	Manager	Every R until acked

Item / ticket ready	Responsible waiter	Pager, waiter app, tablet status	T7 READY	Manager	Every R until acked
Water request	Responsible waiter	Pager, waiter app	T4 WATER	Manager	Every R until acked
Waiter request	Responsible waiter	Pager, waiter app	T4 WAITER	Manager	Every R until acked
Bill request	Responsible waiter + cashier	Pager, waiter app, POS	T4 BILL	Manager	Every R until acked
Ready not collected ("Notify manager" on KDS)	Manager(s) on duty	POS, dashboard, manager pager	T7 FOOD WAITING	-	Every R until acked
Manager nudge	Selected waiter(s)	Pager, waiter app	MGR: <message>	- (ack time recorded)	Every R until acked
Order cancelled / modified	Station(s)	KDS alert + printed slip	-	-	-
Waiter's pager and app both offline	Manager	POS, dashboard	-	Immediate	-
Pager / tablet low battery or offline	Manager (+ wearer for own pager)	Dashboard, waiter app, pager	LOW BATTERY	-	Once per state change

Printer offline	Manager, cashier	POS, dashboard	-	-	Until resolved
Disk $\geq$ 80 % / backup failed	Owner, manager	POS banner, dashboard; reported to Control Plane	-	-	Daily
License state change	Owner, manager	POS banner, dashboard	-	-	Daily

Appendix D: Changes from the original blueprint

Everything in the blueprint is retained unless listed here.

Blueprint	This BRD	Reason
Server bundled inside the Electron POS app	Separate Windows service + watchdog (§ 10.2)	Tablets/pagers must not stop if the POS window closes
Kitchen display as an Electron app	Browser-based KDS mode (KDS-001)	Cheaper hardware; one codebase
Separate manager dashboard website	Same web console as POS, role-based (MGR-001)	Less code; works on PC and phone
PIN-only login	Name tile + PIN, pepper, device pairing (AUTH-001, -002, -007)	4-digit PINs are weak alone; no PIN-uniqueness leak
"Every query filters by restaurant"	Enforced through RLS + CI isolation tests in the cloud; local DB single-tenant but keeps restaurant ID (SEC-005)	The real multi-tenant risk is in the cloud

PubSubClient on the pager	esp-mqtt with TLS + QoS 1; secure boot; signed OTA (PGR)	Reliable acknowledgements and security
@react-native-voice/voice + fuse.js	Maintained speech module + fuzzy + synonyms + tags (TAB-006/007)	"Something spicy" needs tags; library maintenance
License file with cached copy	Rolling 30-day signed license, machine binding, staged states, restricted mode (LIC)	Payment enforcement without harming service or data
Automatic updates	Channels, per-restaurant pinning, maintenance windows, rollback (UPD)	"Fix for a specific restaurant" and safe updates
Heartbeat	Full Vendor Control Plane (VCP)	Operating many restaurants needs a real back office
(not in blueprint)	Waiter approval of customer orders; service-request lifecycle with Cancel; escalation to manager; recommendations; backups/archive/purge; anti-fraud; GST billing; day-end; onboarding wizard; compliance	Business Owner brief and workshop decisions